

Graduation Report

Presented at

The National School of Electronics and Telecommunications of Sfax

In order to obtain the

Engineering Diploma in: Data Engineering and Decisional System

Option :

Business Intelligence

By

Mohamed Guidara

Itqān: An RFQ Lifecycle Intelligence Platform with a Bounded AI Copilot

Presented on 20/ 06/ 2026, before the review panel

Mrs.	Ines Ben Massoud	President
Mrs.	Molka Rekik	Examiner
Mr.	Boukthir Haddar	Academic Supervisor
Mr.	Omar Abid	Industrial Supervisor
Mr.	Omar Baccar	Industrial Supervisor



DEDICATION



With a heart full of gratitude, I first thank God, whose grace carried me through doubt, fatigue, and uncertainty, and gave me the strength to bring this work to its end.

To my beloved parents, Hajer and Naceur,
for the quiet sacrifices, the prayers that walked ahead of me, and the love that held me steady when I could not hold myself. Every page of this work carries something of you.

To my dear brother, Amir, and his wife Yosr,
for being my shelter, my confidants, and the familiar warmth that made even the heaviest days feel lighter.

To my dear sister, Ons, and her husband Omar,
for your tenderness, your faith in me, and the reassurance you gave me whenever my own certainty became fragile.

To all my family and to the friends who became family,
for standing beside me through every uncertainty, every late night, and every small victory.

To all of you, with all my heart,
I dedicate this work.

Mohamed GUIDARA



Acknowledgement

I would first like to express my sincere gratitude to my academic supervisor, **Mr. Boukthir HADDAR**. His thoughtful direction, his availability, and his rigorous feedback shaped this work from its earliest outline to its final form, and the confidence he placed in the project was a lasting source of motivation.

My deep thanks go equally to my professional supervisors at BACAB Consulting, **Mr. Omar ABID** and **Mr. Omar BACCAR**. Their technical insight, their day-to-day mentorship, and the trust they extended to me anchored this project in genuine engineering practice and substantially strengthened both its relevance and its depth.

I am also grateful to the members of the examination committee, **Mrs. Ines BEN MASSOUD**, President of the jury, and **Mrs. Molka REKIK**, Examiner, for the time and care they devoted to reviewing this dissertation. Their attentive reading and considered remarks have meaningfully improved the quality and precision of the present work.

Finally, I wish to extend my heartfelt thanks to the entire teaching and administrative staff of National School of Electronics and Telecommunications of Sfax, as well as to the whole team at BACAB Consulting. Their collective support, their guidance, and their encouragement accompanied this project throughout and contributed greatly to bringing it to completion.



TABLE OF CONTENTS

LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
GENERAL INTRODUCTION	xii
1 General Context and Business Problem	2
INTRODUCTION	3
1.1 Host Organization and Industrial Context	3
1.2 The Estimation Workflow at GHI	4
1.2.1 The RFQ Package and Estimation Workbook	4
1.2.2 The End-to-End Estimation Workflow	4
1.3 Documented Operational Constraints	5
1.3.1 Audit Methodology	5
1.3.2 Reference: The Twelve Documented Constraints	6
1.4 Audit Findings and the Rejection of BOQ Automation	7
1.5 The Strategic Pivot to RFQ Lifecycle Intelligence	8
1.6 Project Identity and Four-Pillar Vision	8
1.6.1 Project Identity	8
1.6.2 The Four-Pillar Target Architecture	9
1.6.3 The Cross-Cutting Intelligence Layer	9
1.6.4 Scope of the Present Work	10
CONCLUSION	10
2 State of the Art	11
INTRODUCTION	12
2.1 Industrial RFQ Systems and the Lifecycle Visibility Gap	12
2.2 Microservices Architecture and Project-Specific Layering	13

2.3	Conversational AI Patterns for Evidence-Grounded Systems	13
2.4	Grounding, Hallucination, and Safety Constraints	14
2.5	Deterministic Document Intelligence for Engineering Artifacts	15
2.6	Positioning of the Proposed Platform	16
	CONCLUSION	17
3	Requirements, Methodology, and Project Management	18
	INTRODUCTION	19
3.1	Stakeholders, Actors, and Access Assumptions	19
3.1.1	Project Stakeholders	19
3.1.2	System Actors	19
3.1.3	Permission Model and IAM Integration	21
3.2	Functional Requirements	22
3.2.1	Product Backlog	23
3.3	Non-Functional Requirements	23
3.4	Use Cases and Activity Diagrams	25
3.4.1	System-Level Use Cases	25
3.4.2	Activity Diagram: Advance RFQ Stage	26
3.4.3	Conversational Flow with the Copilot	27
3.5	Methodology	27
3.5.1	Scrumban as the Iteration Framework	28
3.5.2	Architectural Decision Records	29
3.5.3	Project Control Center	29
	CONCLUSION	30
4	Conceptual and Technical Architecture	31
	INTRODUCTION	32
4.1	Global Architecture and Four-Pillar Layering	32
4.1.1	The Four Architectural Layers and the Cross-Cutting Copilot	32
4.1.2	The Four Pillars as an Architectural Map	33
4.2	Microservices Inventory and BACAB Pattern	34
4.2.1	Service Inventory	34
4.2.2	The BACAB Layered Pattern	35
4.2.3	Inter-Service Communication	36
4.3	Manager Service: Domain Model and Lifecycle	37
4.3.1	Mission and Domain Model	37

4.3.2	Workflow and Stage Progression	38
4.4	Copilot Service: Trust-Boundary Architecture	39
4.4.1	Design Philosophy	39
4.4.2	Positioning Against Alternative Conversational AI Patterns	40
4.4.3	The Eight Trust Boundaries	40
4.4.4	Two Classification Sources, One Plan Factory	42
4.4.5	The Path Registry as Policy-as-Configuration	43
4.4.6	The Pipeline and the Five Evidence Sources	43
4.4.7	Why This Architecture Represents the Project's Primary Innovation	43
4.5	Evidence Boundary	44
4.6	Platform Integration and ADR Summary	44
4.6.1	Platform Integration	44
4.6.2	Architectural Decisions	45
	CONCLUSION	45
5	Implementation	46
	INTRODUCTION	47
5.1	Development Environment and Toolchain	47
5.1.1	BACAB Discipline Across the Backend Services	49
5.2	Manager Service Implementation	49
5.2.1	BACAB Topology within the Manager Service	50
5.2.2	Relational Schema and Stage Progression	50
5.2.3	Subtasks, Notes, Files, and Reminders	51
5.2.4	Analytics and Telemetry Endpoints	52
5.2.5	Observability and Audit-History Implementation	52
5.3	Copilot Service Implementation	53
5.3.1	Turn Processing Pipeline and Vertical Slicing	53
5.3.2	Path Registry and Single-Construction Invariance	54
5.3.3	Memory, Guardrails, and the Escalation Gate	54
5.4	Intelligence Service Implementation	56
5.5	User Interface Integration	56
5.6	Cross-Service Integration Status	57
	CONCLUSION	57
6	Validation, Results, and Discussion	58
	INTRODUCTION	59

6.1	Validation Strategy and Five Evidence Classes	59
6.2	Manager Service Validation	60
6.3	Copilot Service Validation	61
6.4	Trust-Boundary Verification via Anti-Drift Constraints	62
6.5	Intelligence Service Validation	63
6.6	Frontend and Cross-Service Integration Validation	63
6.7	Business Impact Mapping	64
6.8	Limitations and Future Work	64
	CONCLUSION	65
	GENERAL CONCLUSION	66
	BIBLIOGRAPHY	67
	APPENDIX	71
A.1	Detailed Use Case Descriptions	71
A.1.1	UC-01, Register RFQ	71
A.1.2	UC-02, Advance RFQ Stage	71
A.1.3	UC-03, Manage Stage Artifacts	71
A.1.4	UC-04, Configure Reminders	72
A.1.5	UC-05, Consult Pipeline Analytics	72
A.1.6	UC-06, Query Copilot in Portfolio Mode	72
A.1.7	UC-07, Query Copilot in RFQ-Bound Mode	72
A.1.8	UC-08, Manage Workflow Templates	73
A.2	Product Backlog	73
A.3	Path Registry Structure	74
A.4	Anti-Drift Tests and Test Invocation Evidence	75
A.4.1	Anti-Drift Test Files (Copilot Service)	75
A.4.2	Test Invocations and Pass Counts	76
A.4.3	Frontend Source-Contract Scripts	76
A.5	Golden Scenario Test Set	76
A.6	Postman Validation Structure	76
A.7	Integration Paths	77
A.8	Business Impact Mapping	77
A.9	Limitations Register	78
A.10	Architectural Decisions Register	79



LIST OF FIGURES

1.1	Current RFQ Lifecycle Management Process at GHI	5
1.2	Itqan Platform Logic	10
3.1	Activity diagram for <i>Advance RFQ Stage</i>	27
3.2	Scrumban six-state workflow board, utilizing <i>Documented</i> as the terminal validation column	29
4.1	Global architecture of the Itqan platform: mapping of the four layers, microservices, and the cross-cutting RFQ Copilot	33
4.2	The BACAB layered architecture: structural separation of concerns via active and supporting layers	36
4.3	Entity-relationship diagram detailing the domain model of the manager service	38
4.4	Canonical processing pipeline of the RFQ Copilot, demonstrating the strict encapsulation of LLM invocations within deterministic safeguards	42
5.1	The Escalation Gate and the Path 8 safety family: any stage-failure trigger is deterministically mapped, via a single re-entry into the ExecutionPlanFactory, to one of five Path 8.x safe-degradation responses, with the reason code selecting the rendered template	55
5.2	Representative UI dashboard interface demonstrating the integration of the lifecycle orchestration panel and the copilot conversational drawer	57



LIST OF TABLES

1.1	The twelve documented operational constraints, classified by finding type, with their organizational consequence	6
3.1	System actors of Itqan with their responsibilities and implementation status . .	20
3.2	Target permission model for currently exercised actors	21
3.3	Non-functional requirements grouped by category, indicating current scope and implementation status	23
3.4	Actor-use-case responsibility matrix indicating access scope per role	26
4.1	Operational scope of the four architectural pillars within the present project phase	34
4.2	The eight deterministic trust boundaries defining the copilot's architecture . . .	41
5.1	Principal technological stack utilized for the implementation of the Itqan platform	48
5.2	Implementation evidence synthesis across delivered platform components (test counts derived from the project snapshot detailed in Appendix A.4)	49
6.1	Consolidated aggregation of empirical validation evidence	60
6.2	Validation evidence matrix for the manager service	61
6.3	Taxonomic breakdown of the copilot service automated test suite	61
6.4	Methodological mapping of trust boundaries to their specific enforcement layers	62
6.5	Synthesis of business impact mapping against initial operational constraints . .	64
6.6	Consolidated register of current platform constraints and projected mitigations .	65
A.1	Product backlog mapping functional requirements to current statuses	73
A.2	Automated pytest execution metrics (Commit 72d5ae2, Python 3.11.9)	76
A.3	Cross-service integration mapping and validation maturity	77
A.4	Extensive mapping of audit vulnerabilities to implemented capabilities	77
A.5	Register of architectural limitations and mitigation trajectories	78
A.6	Consolidated registry of foundational architectural decisions	79



LIST OF ABBREVIATIONS

ADR Architectural Decision Record

AI Artificial Intelligence

API Application Programming Interface

ASME American Society of Mechanical Engineers

BACAB Host firm and the internal layered architecture convention used by it across its engagements (routes / controllers / datasources / translators / connectors)

BI Business Intelligence

BOM Bill of Materials

BOQ Bill of Quantities

BPM Business Process Management

CI Continuous Integration

CPQ Configure, Price, Quote

CRUD Create, Read, Update, Delete

DDD Domain-Driven Design

ERD Entity-Relationship Diagram

ERP Enterprise Resource Planning

FR Functional Requirement

LIST OF ABBREVIATIONS

GHI Gulf Heavy Industries

HTTP HyperText Transfer Protocol

IAM Identity and Access Management

IB Internal Bid (RFQ classification)

IDSD Data Engineering and Decisional Systems

IF Internal Fabrication (RFQ classification)

JSON JavaScript Object Notation

KPI Key Performance Indicator

LLM Large Language Model

MR Material Requisition

MS Microservice

NFR Non-Functional Requirement

ORM Object-Relational Mapping

PFE Projet de Fin d'Études (Final Year Project)

PP Pain Point

QC Quality Control

RAG Retrieval-Augmented Generation

REST Representational State Transfer

RFQ Request for Quotation

RVL Restricted Vendor List

SAES Saudi Aramco Engineering Standard

LIST OF ABBREVIATIONS

SAMSS Saudi Aramco Materials System Specification

SDK Software Development Kit

SQL Structured Query Language

SSO Single Sign-On

TB Trust Boundary

TEMA Tubular Exchanger Manufacturers Association

UI User Interface

UML Unified Modeling Language

WIP Work In Progress



GENERAL INTRODUCTION

In industrial manufacturing, a Request for Quotation (RFQ) constitutes a complex, multidisciplinary engineering and operational process. It serves as the entry point of a prolonged lifecycle that requires the rigorous coordination of engineering, production, quality assurance, and procurement departments. The outcome of this process determines whether a project is acquired under viable, profitable terms or results in contractual vulnerability. The present work addresses not the arithmetic generation of the quotation itself, but the structural resilience of the surrounding process: namely coordination, operational visibility, capitalization on historical data, and the early identification of feasibility risks. These elements distinguish a systematic estimation framework from ad-hoc, individual-driven workflows lacking systemic support.

This graduation project was carried out within BACAB Consulting, an engineering consultancy specializing in cloud architecture, microservices, artificial intelligence, and data engineering. The study is grounded in a comprehensive on-site audit conducted at Al Bassam Group, a major industrial conglomerate in Dammam, Saudi Arabia, and its subsidiary GHI (Gulf Heavy Industries Co.), a leading heavy steel fabrication enterprise in the Middle East. The audit identified twelve critical operational constraints, categorized into five structural domains: process coordination, visibility, capitalization of historical data, feasibility and supplier risk management, and tooling resilience.

The initial mandate consisted of developing an artificial intelligence system for the automated generation of Bills of Quantities (BoQ). However, the audit revealed significant challenges: the tacit expertise embedded in the estimation workbooks was not readily extractable as structured training data, and the asymmetric cost of error inherent to industrial quotation precluded the deployment of a fully automated pricing system without

intolerable financial risk. Consequently, the project was strategically reoriented to design a decision-support platform that fortifies all process stages independent of irreducible human expertise, while integrating with manual expert validation.

The resulting platform, designated as *Itqan* (denoting mastery and precision), is a comprehensive web-based system. It manages the RFQ lifecycle, centralizes process artifacts, provides analytical dashboards, deterministically parses engineering documents, and features a constrained conversational AI copilot grounded strictly in authorized platform evidence. The system architecture is predicated on four foundational pillars: Workflow and Tracking, Communication Automation, Business Intelligence, and Predictive Analytics. It is augmented by a cross-cutting conversational layer, the *RFQ Copilot*. The primary engineering contribution lies in the trust-boundary architecture, which strictly governs the copilot's behavior in response to the asymmetric cost of error in the estimation domain. Within the scope of this project, the operational workflow-tracking backbone of the first pillar is implemented, while the second and third are partially addressed. The fourth pillar is established at the foundational level via deterministic document parsing, with predictive modeling deferred to future iterations.

The report details the project across six chapters. Chapter 1 presents the organizational context, the industrial framework, the audit findings, and the resulting strategic reorientation. Chapter 2 situates the proposed platform within the current technological and academic state of the art. Chapter 3 formalizes the system actors, functional and non-functional requirements, and the development methodology. Chapter 4 details the conceptual and technical architecture, highlighting the trust-boundary architecture as the central innovation. Chapter 5 discusses the implementation of the platform components: the manager service, the copilot service, the intelligence service, and the user interface. Finally, Chapter 6 evaluates the system and verifies, within the documented evidence scope, that the architectural invariants are preserved under empirical evaluation. The General Conclusion summarizes the contributions and outlines perspectives for future work.

General Context and Business Problem

Contents

INTRODUCTION	3
1.1 Host Organization and Industrial Context	3
1.2 The Estimation Workflow at GHI	4
1.2.1 The RFQ Package and Estimation Workbook	4
1.2.2 The End-to-End Estimation Workflow	4
1.3 Documented Operational Constraints	5
1.3.1 Audit Methodology	5
1.3.2 Reference: The Twelve Documented Constraints	6
1.4 Audit Findings and the Rejection of BOQ Automation	7
1.5 The Strategic Pivot to RFQ Lifecycle Intelligence	8
1.6 Project Identity and Four-Pillar Vision	8
1.6.1 Project Identity	8
1.6.2 The Four-Pillar Target Architecture	9
1.6.3 The Cross-Cutting Intelligence Layer	9
1.6.4 Scope of the Present Work	10
CONCLUSION	10

INTRODUCTION

Building upon the framework established in the General Introduction, this chapter details the empirical evidence motivating the project. It presents the host organization and the industrial context at Al Bassam Group, describes the estimation workflow as observed during the audit, catalogues the twelve documented operational constraints, explains the rationale for rejecting the original Bill of Quantities (BoQ) automation mandate, and outlines the four-pillar architectural vision of the Itqan platform.

1.1 Host Organization and Industrial Context

This research was conducted as a final-year engineering project within BACAB Consulting, a Tunisia-based consulting firm specializing in cloud architecture, microservices, artificial intelligence, and data engineering. The practical foundation consisted of a five-day on-site audit at Al Bassam Group, a major industrial conglomerate based in Dammam, Saudi Arabia, and its subsidiary GHI (Gulf Heavy Industries Co.) in Jubail Industrial City. GHI is a leading heavy steel fabrication enterprise in the Middle East, manufacturing pressure vessels, heat exchangers, and related equipment for the oil-and-gas, petrochemical, and energy sectors, notably serving Saudi Aramco. The audit engaged the estimation department, executive management, as well as finance and engineering personnel whose roles intersect with the estimation process. Active development commenced in January 2026 following the consolidation of the audit findings.

Industrial estimation in this domain fundamentally differs from generic commercial estimation: the artifacts are highly technical, the products are governed by layered international standards, multiple engineering disciplines are required to validate pricing, and the manufacturer assumes long lead times and substantial contractual liabilities upon bid acceptance. Consequently, the surrounding workflow, rather than the arithmetic pricing, constitutes the primary source of operational risk.

1.2 The Estimation Workflow at GHI

1.2.1 The RFQ Package and Estimation Workbook

Upon receipt at GHI, a Request for Quotation (RFQ) typically constitutes a complex, structured package centered around a Material Requisition (MR), which serves as the canonical reference throughout the project lifecycle. A representative MR package observed during the audit comprised approximately fifty-five technical documents organized according to the canonical GHI taxonomy: scope of supply, bill of materials, vendor restrictions, technical specifications, engineering drawings, applicable standards, and quality assurance requirements. Three core artifacts dictate the majority of the estimation effort: the Bill of Materials (BOM), the Restricted Vendor List (RVL), and the corpus of applicable standards. This corpus includes the Saudi Aramco Materials System Specification (SAMSS) and Saudi Aramco Engineering Standard (SAES), alongside international frameworks from the American Society of Mechanical Engineers (ASME) [16] and the Tubular Exchanger Manufacturers Association (TEMA) [17].

Following triage, the estimation team formulates a quotation by constructing an estimation workbook: a unified Microsoft Excel file based on the GHI template. This involves a three-sheet active anchor set (General, Bid S, Top Sheet) mapped to a thirty-two-sheet source workbook (represented by the reference fixture IF-25144). This workbook captures all cost drivers associated with fabrication, computing values concurrently in Saudi Riyals, US Dollars, percentage share, and cost per ton.

1.2.2 The End-to-End Estimation Workflow

The RFQ lifecycle at GHI comprises six primary phases (Figure 1.1). **Intake and triage:** the RFQ is received via email or the SAP Ariba portal, followed by a preliminary feasibility assessment by the department manager (requiring ten to twenty minutes, with an observed acceptance rate of approximately ninety percent). **Registration and classification:** the RFQ is

allocated a unique identifier and categorized as either Internal Bid (IB) or Internal Fabrication (IF), dictating the subsequent workflow. **Design:** the engineering department leverages prior designs or generates a preliminary design, consolidating the detailed BOM. **Cost estimation:** the estimation workbook is populated across its source sheets. **Fabrication decision:** management determines whether execution will be internal or subcontracted. **Submission and follow-up:** the bid undergoes internal approval, is transmitted to the client, and tracked through three potential outcomes (won, lost, pending). This workflow requires coordination across distinct departments, with Estimation acting as the central hub, alongside Engineering, Production, Quality Control, and Procurement. The systemic fragility introduced by this cross-departmental coordination motivated the audit findings detailed in the subsequent section.

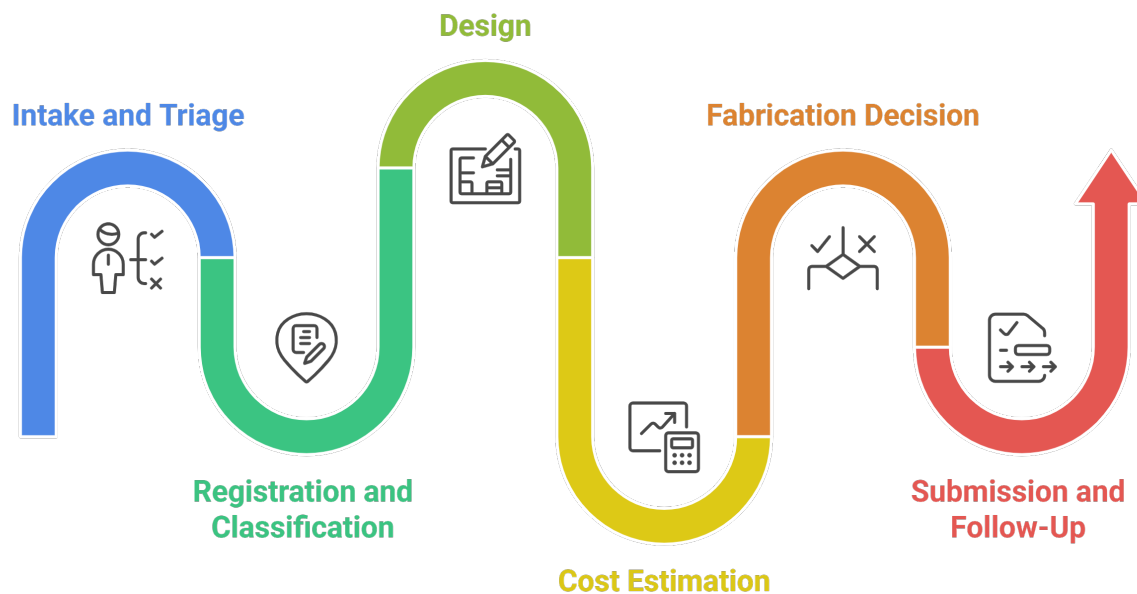


Figure 1.1: Current RFQ Lifecycle Management Process at GHI

1.3 Documented Operational Constraints

1.3.1 Audit Methodology

The audit was conducted over five days in late November, alternating between the locations introduced in Section 1.1. The methodology synthesized three complementary data sources: structured interviews focusing on responsibilities, daily workflows, and recurring operational

friction; direct observation of process artifacts, notably the estimation workbook and RFQ documentation; and a retrospective analysis of historical projects to identify systemic failure points.

1.3.2 Reference: The Twelve Documented Constraints

The audit yielded twelve documented constraints classified by finding type: *audit findings* directly reported by GHI stakeholders or observed in process artifacts, and *analytical findings* derived during post-audit synthesis. Table 1.1 assigns stable identifiers (PP-01 to PP-12) utilized throughout the report, notably in Section 1.5 and Chapter 6.

ID	Finding Type	Title	Consequence
PP-01	Audit finding	Inter-departmental communication	Cross-department information failures.
PP-02	Audit finding	Manual client follow-up	Missed deadlines resulting from manual tracking.
PP-03	Audit finding	Lack of executive visibility	Absence of real-time monitoring for the estimation pipeline.
PP-04	Analytical finding	Inadequate reuse of past RFQs	Institutional knowledge lost per new RFQ.
PP-05	Audit finding	Late discovery of infeasibility	Critical feasibility blockers identified too late in the process.
PP-06	Analytical finding	Workbook as single point of failure	Critical financial model lacks version control and robust validation.
PP-07	Analytical finding	Undocumented feasibility expertise	Managerial go/no-go reasoning is not formalized.
PP-08	Analytical finding	Classification not enforcing workflow	IB/IF distinction fails to drive procedural gates.
PP-09	Audit finding	Unmanaged supplier risk	Absence of systematic tracking for reliability or lead times.
PP-10	Audit finding	Underutilized ERP system	Odoo system disconnected from the estimation workflow.
PP-11	Audit finding	Heuristic-based bid prioritization	Decision-making lacking probabilistic win analysis.
PP-12	Analytical finding	Standards not driving early cost	ASME/TEMA financial implications recognized late.

Table 1.1: The twelve documented operational constraints, classified by finding type, with their organizational consequence

1.4 Audit Findings and the Rejection of BOQ Automation

Collectively, these twelve operational constraints highlight a systemic deficiency: the lack of a structured, observable, and traceable representation of the RFQ lifecycle accessible across the organization. While the domain expertise is mature and standard compliance is well understood, the process lacks the integrative framework required to transition from isolated expert actions to a cohesive, coordinated workflow. This diagnosis was presented in response to the original technical request from Al Bassam Group, which sought an artificial intelligence system to automate the generation of the Bill of Quantities (BoQ). The audit findings demonstrated that such a mandate was fundamentally misaligned with the actual operational bottlenecks.

The original BoQ mandate presumed that recent advancements in Large Language Models (LLMs) and structured data extraction were mature enough to encode complex estimation expertise into a generative system. The audit firmly rejected this premise based on two structural observations.

First: The expertise embedded in the workbook is largely tacit. Senior estimators populate the sheets based on implicit reasoning concerning material behavior, supplier reliability, the fabrication complexity of specific geometries, and the strategic contractual posture suited for a particular client. This reasoning cannot be extracted solely from the workbook data; the workbook represents the output of the estimation process, not the cognitive specification.

Second: The cost of error is highly asymmetric. In multi-million Saudi Riyal quotations, an underestimated price locks GHI into a contractual loss potentially exceeding the project margin, whereas an overestimated price results in a lost bid. There is no acceptable failure mode for a fully automated generative system within this regime, nor is there a margin of accuracy that would justify substituting expert estimators given the contractual risks involved.

1.5 The Strategic Pivot to RFQ Lifecycle Intelligence

Rather than diluting the objective into a partial BoQ assistant (where the risks would eclipse the marginal utility), the audit recommended a strategic reorientation. The constraints identified in Section 1.3 do not stem from price calculation inefficiencies; they originate from deficiencies in coordination, visibility, historical knowledge capitalization, feasibility risk management, and tooling resilience. The estimation workbook, despite constituting a fragile single point of failure, is not the primary bottleneck; the bottleneck lies in the unstructured processes surrounding it.

Consequently, the project scope was explicitly redefined: rather than automating components dependent on irreducible human expertise, the objective shifted toward developing a decision-support platform that fortifies all surrounding processes while accommodating manual expert input. The platform is designed to track the comprehensive lifecycle of an RFQ across departments, provide executive leadership with unprecedented visibility, capture currently undocumented feasibility decisions, surface risks earlier, and offer a conversational interface to facilitate interaction with complex artifacts. The critical price calculation remains securely assigned to senior estimators utilizing their established tools. The platform is architected to encapsulate and support this core expertise, not to replace it.

This fundamental pivot shifts the system paradigm: a BoQ automator constitutes a closed-loop generative system evaluated against estimator accuracy; conversely, a lifecycle intelligence platform is an open-loop coordination and decision-support system, evaluated by improvements in process duration, traceability, and overall decision quality.

1.6 Project Identity and Four-Pillar Vision

1.6.1 Project Identity

The platform is designated *Itqan*, derived from the Arabic word *itqān*, signifying mastery, precision, and the thorough execution of work. This nomenclature reflects the project's

core philosophy: in an industrial estimation context where errors carry severe contractual consequences, the objective is not mere acceleration at the expense of rigor. Instead, the goal is to introduce discipline, traceability, and structured intelligence into a workflow historically reliant on individual diligence. Itqan serves as an instrument of process governance rather than disruptive automation.

1.6.2 The Four-Pillar Target Architecture

The constraints outlined in Section 1.3 map to four primary areas of concern, informing the target architecture of Itqan. **Workflow and Tracking** forms the operational backbone: it maintains a structured lifecycle state machine for each RFQ, tracks inter-departmental progress, and provides role-based access control. **Communication Automation** mitigates manual client interaction via structured reminders, deadline monitoring, and a comprehensive audit trail linked to specific RFQs. **Business Intelligence and Executive Visibility** equips leadership with real-time analytics through dedicated endpoints and dashboards, visualizing pipeline value, win rates, client profitability, and process bottlenecks. **Predictive Analytics and Historical Learning** represents the longest-horizon pillar, focusing on the systematic exploitation of historical RFQ data through pattern matching, supplier intelligence, and probabilistic win-estimation. The exact mapping of these pillars to the identified constraints is detailed in Chapter 6 (Section 6.7).

1.6.3 The Cross-Cutting Intelligence Layer

While the four pillars define the functional capabilities of the platform, they do not dictate the user interaction paradigm. Therefore, Itqan incorporates a conversational layer, the **RFQ Copilot**, characterized by strict evidence boundaries and test-backed trust-boundary controls governing data retrieval. The copilot does not manage a specific pillar; rather, it navigates and interprets the structured data generated across all pillars, as illustrated in Figure 1.2. The architectural design and the technical innovations of the copilot are elaborated in Chapter 4.

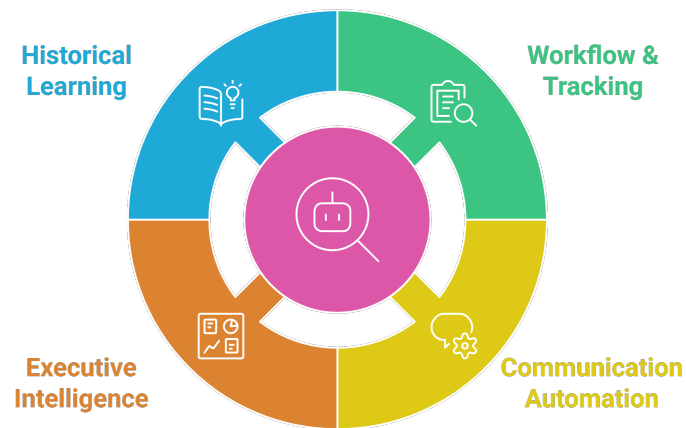


Figure 1.2: Itqan Platform Logic

1.6.4 Scope of the Present Work

Within the scope of this graduation project, the first pillar is implemented as the operational backbone of the platform. The second pillar is partially addressed through a reminder infrastructure and workflow visibility, deferring outbound delivery channels and inbound monitoring to future development. The third pillar is partially supported via analytical endpoints and demonstration dashboards. The fourth pillar is established at a foundational level through deterministic document parsing, with advanced predictive models designated as future scope. The trust-bound conversational copilot represents the central architectural and engineering contribution of this work. The platform serves as a robust foundation, with the realization of its complete vision planned over a multi-year horizon.

CONCLUSION

This chapter has established the empirical audit findings that motivated the development of Itqan, justifying the strategic pivot from generative BoQ automation to comprehensive RFQ lifecycle intelligence. It introduced the platform's objectives and its four-pillar target architecture. The subsequent chapter situates the project within the current technological and academic landscape.

State of the Art and Technological Background

Contents

INTRODUCTION	12
2.1 Industrial RFQ Systems and the Lifecycle Visibility Gap . .	12
2.2 Microservices Architecture and Project-Specific Layering . .	13
2.3 Conversational AI Patterns for Evidence-Grounded Systems	13
2.4 Grounding, Hallucination, and Safety Constraints	14
2.5 Deterministic Document Intelligence for Engineering Artifacts	15
2.6 Positioning of the Proposed Platform	16
CONCLUSION	17

INTRODUCTION

The system proposed in this report operates at the intersection of three technological domains: industrial quotation and Request for Quotation (RFQ) management; microservices and layered architectural patterns; and conversational agents powered by Large Language Models (LLMs). This chapter critically reviews each landscape to identify the specific operational gap the proposed platform addresses, rather than conducting an exhaustive survey of any single field. Section 2.6 synthesizes these findings and defines the strategic positioning of the proposed platform.

2.1 Industrial RFQ Systems and the Lifecycle Visibility Gap

Three categories of commercial software predominantly support industrial quotation. **Enterprise Resource Planning (ERP)-integrated quoting modules**, exemplified by SAP CPQ [14], embed quote generation within the transactional and master-data backbone of the enterprise. **Standalone Configure, Price, Quote (CPQ) systems**, exemplified by Salesforce CPQ [15], focus on configuration, pricing, and proposal generation, offering advanced support for configuration logic and discounting rules. **Custom spreadsheet-based estimation** remains prevalent in domains where the cost model relies heavily on uncodified expert judgment, as exemplified by the GHI estimation workbook discussed in Chapter 1. Business process management literature [5] frames the RFQ activity as a coordinated orchestration spanning multiple roles and decision points, distinct from any isolated price calculation.

Each category optimizes a specific subset of the quotation activity. None of these paradigms adequately address the specific operational gap identified during the Al Bassam Group audit: cross-departmental lifecycle coordination, executive-level visibility, capitalization on historical data, early identification of feasibility risks, and the structured capture of interim decisions. The primary operational bottleneck resides not within the estimation workbook, but in its surrounding lifecycle. The proposed platform is designed to provide the combination of capabilities required to address this lifecycle: RFQ orchestration, deterministic document

intelligence, evidence-grounded conversational access, and a foundational architecture for future predictive learning.

2.2 Microservices Architecture and Project-Specific Layering

At the architectural level, the proposed system is structured as five responsibility-segregated microservices [2], decomposed along the functional boundaries of the RFQ lifecycle: operational lifecycle management, document and workbook intelligence, conversational assistance, user interaction, and identity and access management. This detailed decomposition is presented in Chapter 4 (Section 4.2). Internally, each backend service implements the **BACAB pattern**, a project-specific synthesis of the traditional three-tier architecture and the Ports-and-Adapters paradigm [1], informed by clean-architecture layering principles [3]. This pattern separates entry points (routes), business orchestration (controllers), persistence access (datasources), external communication (connectors), and data representation (translators), supported by models, configuration, and utilities layers. This architecture ensures lifecycle traceability, separates operational source-of-truth from derived intelligence, and facilitates independent service evolution; it is conceptualized in Chapter 4 (Section 4.2) and implemented in Chapter 5 (Section 5.1).

2.3 Conversational AI Patterns for Evidence-Grounded Systems

The conversational layer of the proposed platform leverages the recent advancements in Large Language Models [8]. The four design patterns reviewed below share this foundational technology; however, they differ significantly in the constraints they impose on model behavior. The critical objective is determining the optimal pattern for an industrial context characterized by asymmetric costs of error.

Naive Retrieval-Augmented Generation (RAG) [6] appends retrieved documents to the prompt; while suitable for question-answering over static corpora, this pattern lacks structural defenses to prevent the model from hallucinating beyond the retrieved context. **LLM agents with autonomous tool calling**, exemplified by ReAct [7], grant the model autonomy over tool selection and execution sequence. While appropriate for exploratory tasks, this autonomy poses a structural risk in environments where retrieved values inform binding contractual quotations. **Constrained tool-calling chatbots** leveraging provider function-calling APIs [12] retain execution control within the application code, yet still allow the model to influence policy via intent classification. **Trust-bounded or policy-as-configuration systems**, the emerging fourth paradigm, delegate all safety-critical decisions (e.g., tool authorization, data access control, refusal triggers, context injection) to deterministic platform components rather than the probabilistic language model. While a canonical designation for this architectural family remains to be standardized, it represents the architectural commitment of the proposed platform, justified by the safety and grounding imperatives developed in Section 2.4 and operationalized in Chapter 4 (Section 4.4).

2.4 Grounding, Hallucination, and Safety Constraints

The deployment of any language-model-based conversational agent necessitates the mitigation of **hallucination**: the generation of factually incorrect yet locally coherent and stylistically plausible content. The survey by Ji et al. [9] establishes hallucination as an intrinsic structural property of the generative paradigm rather than a transient anomaly; consequently, mitigation mechanisms must be engineered into the surrounding system architecture rather than expected to emerge from the model inherently. A corollary constraint is **context dilution**, wherein relevant evidence is obfuscated by extraneous material within an extensive prompt. This constraint necessitates the implementation of the per-path field whitelist developed in Chapter 4 (Section 4.5).

While Retrieval-Augmented Generation mitigates hallucination, it does not eradicate it; the model remains susceptible to drifting beyond the provided evidence. A system relying

exclusively on RAG provides insufficient structural defense against such drift. Therefore, the proposed copilot integrates four complementary mechanisms: **deterministic guardrails** evaluate the generated response prior to delivery (enforcing evidence grounding, scope adherence, and shape conformance); an **LLM-as-Judge layer** [11] executes final linguistic verification (detecting fabrication, scope drift, and forbidden inferences) that deterministic rules cannot reliably capture; **structured outputs** [13] constrain the model to generate strictly schema-conformant JSON; and **policy-as-configuration** [10] encodes safety policies as declarative data rather than procedural code, enabling auditable scope modifications without necessitating pipeline redeployments. This combination forms the foundation of the trust-boundary architecture, conceptualized in Chapter 4 (Section 4.4) and validated in Chapter 6 (Section 6.4).

2.5 Deterministic Document Intelligence for Engineering Artifacts

The estimation workbook and Material Requisition package, introduced in Chapter 1, are structured artifacts whose data parameterizes the cost models. Two primary paradigms exist for extracting information from such documents. **Deterministic parsing** leverages inherent document structure: spreadsheets with predefined layouts are parsed via cell coordinates, and documents with standardized section hierarchies are parsed by identifiers, yielding structured data without generative interpretation. Mature parsing ecosystems exist for standard formats. **LLM-based document question answering** submits document content to a language model to extract specific values. While highly adaptable to document variability, this approach is inherently vulnerable to hallucination (Section 2.4), a critical flaw when extracting numeric values that cannot be readily verified visually by users.

The architectural selection between these paradigms is dictated by the cost-of-error posture (Chapter 1): an erroneous value propagated into a binding quotation incurs significantly higher financial risk than a missing value that safely triggers a manual review. Consequently, the

intelligence service adopts a **deterministic-parsing-first strategy**: parsers extract structured profiles, and values failing extraction thresholds are flagged as anomalies requiring human intervention rather than being probabilistically approximated. This design principle is the central contribution of the document-intelligence layer, demonstrated by the parser implementations detailed in Chapter 5 (Section 5.4).

These artifacts are strictly governed by industrial standards. Pressure vessel construction complies with the ASME Boiler and Pressure Vessel Code, Section VIII Division 1 [16]; shell-and-tube heat exchangers adhere to the standards of the Tubular Exchanger Manufacturers Association [17]. The intelligence service's parsers are engineered to systematically extract standard references within the Material Requisition package, providing a data foundation that supports the later identification of standards-driven cost implications, addressing a critical operational delay identified during the audit (PP-12).

2.6 Positioning of the Proposed Platform

The preceding sections have reviewed the systemic categories against which the proposed platform is positioned: ERP-integrated quoting modules, standalone CPQ systems, custom spreadsheet-based estimation, the four paradigms of LLM-powered conversational architectures, and document-intelligence methodologies. These categories are not mutually exclusive competitors; rather, each addresses a distinct facet of the industrial quotation process. The core contribution of the proposed platform resides in synthesizing capabilities tailored to the specific operational gap identified during the audit, rather than claiming superiority in domains natively addressed by incumbent systems.

The proposed platform is formally positioned as an **RFQ Lifecycle Intelligence Platform integrating a trust-bounded conversational layer**. It synthesizes lifecycle awareness derived from Business Process Management (BPM) principles (Section 2.1) [5]; multi-actor coordination structured via Domain-Driven Design [4] (Chapter 4, Section 4.3); robust evidence grounding mechanisms addressing safety constraints [9, 10, 11]; strict scope-bounded refusal enforced by structured-output discipline [13] and deterministic

guardrails; policy-as-configuration auditability characteristic of trust-bounded architectures; and a risk-averse, deterministic-parsing-first strategy for engineering documents. Chapter 4 formulates the architecture enabling this synthesis; Chapter 5 details its technical implementation; and Chapter 6 validates the architectural invariants through empirical evaluation.

CONCLUSION

This chapter has evaluated the three technological landscapes pertinent to the proposed platform, isolating the operational gaps each paradigm leaves unresolved in the context of the audit findings. The platform is strategically positioned within this gap as an RFQ Lifecycle Intelligence Platform featuring a trust-bounded conversational layer; it does not aim to replace ERPs, compete with CPQs, act as a generic chatbot, or function as a fully autonomous BoQ generator. The specific architecture realizing this strategic positioning is detailed in the subsequent chapter.

Requirements, Methodology, and Project Management

Contents

INTRODUCTION	19
3.1 Stakeholders, Actors, and Access Assumptions	19
3.1.1 Project Stakeholders	19
3.1.2 System Actors	19
3.1.3 Permission Model and IAM Integration	21
3.2 Functional Requirements	22
3.2.1 Product Backlog	23
3.3 Non-Functional Requirements	23
3.4 Use Cases and Activity Diagrams	25
3.4.1 System-Level Use Cases	25
3.4.2 Activity Diagram: Advance RFQ Stage	26
3.4.3 Conversational Flow with the Copilot	27
3.5 Methodology	27
3.5.1 Scrumban as the Iteration Framework	28
3.5.2 Architectural Decision Records	29
3.5.3 Project Control Center	29
CONCLUSION	30

INTRODUCTION

Chapter 1 established the empirical audit findings and the resulting strategic reorientation; Chapter 2 positioned the proposed platform relative to existing technological paradigms. This chapter formalizes the engineering framework that structures the project: the stakeholders, system actors, and access control models; the functional and non-functional requirements derived from the audit; the systemic use cases and activity flows; and the agile methodology adopted to mitigate the constraints of a single-developer environment.

3.1 Stakeholders, Actors, and Access Assumptions

The platform's ecosystem comprises two distinct entities: *stakeholders*, whose strategic expectations define the project's criteria for success, and *actors*, representing the functional roles interacting with the system via its user interfaces, APIs, or the conversational layer.

3.1.1 Project Stakeholders

Four primary stakeholders govern this project: the academic supervisor at ENET'COM, the professional supervisor at BACAB Consulting, the technical and executive leadership at Al Bassam Group, and the academic evaluation jury.

3.1.2 System Actors

The platform's interaction model defines five primary actors: three are operational within the current scope, while two are reserved for future systemic extensions. Conversational interactions via the RFQ Copilot are not modeled as an independent actor; rather, the copilot acts as a cross-cutting capability utilized by the business actors, with its access scope strictly bound to the authenticated user's privileges. Table 3.1 details this actor catalog.

Actor	Role and Responsibilities	Status
Estimation Manager	Exercises operational governance over the RFQ lifecycle: intake, feasibility assessment, classification, workflow orchestration, stage supervision, and final bid approval. This role mirrors the orchestration function observed within GHI's estimation department.	Current
Estimator	Executes granular cost estimation tasks on assigned RFQs: populates the estimation workbook, associates technical artifacts and notes to relevant lifecycle stages, and advances the RFQ through the formal estimation phase.	Current
Executive	Accesses portfolio-level Key Performance Indicators (KPIs) and strategic decision-support summaries via analytical endpoints and dashboards, without executing operational mutations on individual RFQs.	Current
Departmental User	A generalized actor representing the Engineering, Production, Quality, and Procurement functions identified during the audit (Chapter 1). Each provides stage-specific inputs and resolves departmental blockers.	Future
Administrator	Administers workflow templates and identity management policies, contingent upon the deployment of a dedicated Identity and Access Management (IAM) service.	Future

Table 3.1: System actors of Itqan with their responsibilities and implementation status

3.1.3 Permission Model and IAM Integration

Each actor is bound by a strict permission scope defining RFQ visibility, permitted lifecycle mutations, and authorized analytical surfaces. Table 3.2 summarizes the target permission model for the active actors. The future actors will inherit expanded permissions upon the delivery of the dedicated identity service.

Actor	Lifecycle Operations	Analytics Surface	Copilot Scope
Estimation Manager	Full read and write privileges across the departmental portfolio; holds authority for classification, approval, and stage supervision.	Comprehensive portfolio analytics.	Portfolio mode and RFQ-bound mode across all departmental RFQs.
Estimator	Read and write privileges strictly limited to assigned RFQs; authorized to advance assigned RFQs through estimation stages.	Read-only access to personal pipeline metrics.	RFQ-bound mode on assigned RFQs; portfolio mode scoped exclusively to assigned RFQs.
Executive	Read-only access to portfolio aggregates; prohibited from executing state mutations on individual RFQs.	Comprehensive portfolio analytics and strategic summaries.	Portfolio mode only.

Table 3.2: Target permission model for currently exercised actors

In the current implementation, this permission model is enforced via *actor-context propagation*: every inbound request carries an actor context in the current prototype, which the manager service uses to scope data retrieval and protect state-mutating operations. The copilot inherits this exact scope rather than maintaining an independent authorization matrix, as every evidence retrieval request is routed through the manager service’s scoped endpoints (FR-COP-06, NFR-SEC-02). A dedicated *Identity and Access Management (IAM) service* is architecturally reserved as a future integration point within the manager (Authorization-header forwarding and an identity-resolution connector, with token validation delegated to the future service). Comprehensive cross-tenant isolation and granular role-based access control (RBAC) are deferred to this future IAM service (NFR-SEC-01, NFR-SEC-04).

3.2 Functional Requirements

The functional requirements delineate the mandatory operational capabilities provided to the actors defined in Section 3.1. These requirements are derived directly from the operational constraints documented in Chapter 1; each constraint informs one or more specific requirements. They are categorized by system module and assigned unique identifiers for traceability against the validation evidence in Chapter 6.

The twenty-five functional requirements are distributed across the four platform modules and cross-cutting domains. Eight **Manager-service** requirements (FR-MGR-01 to FR-MGR-08) govern RFQ registration with classification metadata, workflow-driven stage instantiation, three-gate stage progression (mandatory-field, blocker, transition-validity), artifact association with append-only note semantics, manual and rule-based reminder automation, analytical aggregations (status distributions, win-rates, client-specific metrics), lifecycle traceability via non-destructive update semantics, and cascading soft-deletion for subtasks. Eight **Copilot-service** requirements (FR-COP-01 to FR-COP-08) define evidence-grounded natural-language generation, dual contextual entry points (portfolio vs. RFQ-bound), deterministic turn classification, path-authorized evidence retrieval (currently bounded to manager state, extensible to intelligence artifacts), structured refusal mechanisms for out-of-scope queries, manager-mediated access enforcement, bounded working-memory management, and strict pre-delivery response verification. Four **Intelligence-service** requirements (FR-INT-01 to FR-INT-04) dictate deterministic parsing of MR-packages against canonical layouts, estimation-workbook parsing against the GHI template (*General*, *Bid S*, *Top Sheet* anchor set), intelligence briefing generation, and structured anomaly reporting over silent error suppression. Three **User-Interface** requirements (FR-UI-01 to FR-UI-03) specify the portfolio dashboard, detailed RFQ views, and role-aware executive indicator surfaces. Two **Cross-cutting** requirements (FR-X-01, FR-X-02) mandate independent backend containerization and standardized health and observability endpoints.

The comprehensive product backlog, detailing priority levels and implementation statuses, is documented in Appendix A.2 (Table A.1). Detailed implementation and validation coverage for each requirement are analyzed in Chapters 5 and 6.

3.2.1 Product Backlog

The aforementioned functional requirements are managed operationally as a 25-item prioritized product backlog (Appendix A.2). The dedicated IAM service is tracked as a deliberately deferred item outside this active backlog, as established in Chapter 1.

3.3 Non-Functional Requirements

The non-functional requirements encompass standard production-grade constraints alongside specific imperatives for an LLM-based conversational layer. They are categorized into five domains: Reliability (REL), Security (SEC), AI Integrity (AI), Maintainability (MNT), and Operability (OPS). Each requirement is tagged with a Scope/Status indicator distinguishing fully implemented attributes from partial or deferred components (Table 3.3).

Table 3.3: Non-functional requirements grouped by category, indicating current scope and implementation status

ID	Category	Requirement	Scope / Status
NFR-REL-01	Reliability	State-mutating operations on lifecycle records shall preserve traceability via timestamps and non-destructive update semantics; a dedicated audit-log surface represents a hardening direction.	Implemented; audit-log hardening direction
NFR-REL-02	Reliability	The system shall fail safely during transient infrastructure anomalies, preserving persisted state and returning structured error payloads to facilitate safe re-execution.	Partial; safe-failure implemented, automatic retry deferred
NFR-REL-03	Reliability	Records flagged for soft deletion shall remain accessible for auditing; no destructive deletion shall execute on critical lifecycle data.	Implemented

continued on next page

Table 3.3 — continued from previous page

ID	Category	Requirement	Scope / Status
NFR-REL-04	Reliability	State mutations and errors within the manager service shall propagate a structured correlation identifier to support distributed tracing.	Partial; manager-internal implemented, cross-service deferred
NFR-SEC-01	Security	Role-Based Access Control (RBAC) shall be enforced at the service layer independently of the client. Currently exercised via actor-context headers and manager-mediated retrieval; full IAM integration is deferred.	Partial; IAM integration point built, identity service deferred
NFR-SEC-02	Security	The copilot service shall explicitly not bypass manager access controls: all evidence retrieval must route through authorized manager endpoints.	Implemented
NFR-SEC-03	Security	Authentication credentials and external API keys shall be strictly excluded from application logs, HTTP responses, and persistence layers.	Implemented as coding discipline; automated redaction tests deferred
NFR-SEC-04	Security	Database schemas and queries shall ensure per-user data scoping; absolute cross-tenant data isolation is contingent upon the deferred IAM service.	Partial; IAM deferred
NFR-AI-01	AI Integrity	The copilot shall not generate factual claims regarding RFQs, lifecycle states, or platform data unsupported by evidence retrieved during the current computational turn.	Implemented (Path 4)
NFR-AI-02	AI Integrity	The copilot shall explicitly justify its responses by referencing the retrieved evidence, ensuring claim verifiability.	Implemented (Path 4)
NFR-AI-03	AI Integrity	The copilot shall execute a structured refusal, rather than a probabilistic approximation, for queries outside its defined scope or lacking sufficient evidence.	Implemented
NFR-AI-04	AI Integrity	Authority over operational truth, evidence selection, and access enforcement shall reside exclusively within deterministic platform components, not the language model.	Implemented
NFR-MNT-01	Maintainability	Services shall expose capabilities strictly through versioned, documented API contracts; internal data structures shall remain encapsulated within service boundaries.	Implemented

continued on next page

Table 3.3 — continued from previous page

ID	Category	Requirement	Scope / Status
NFR-MNT-02	Maintainability	Backend services shall implement the BACAB layered architecture (routes, controllers, datasources, supported by translators, models, configuration, and utilities).	Implemented (backend services)
NFR-MNT-03	Maintainability	Major architectural decisions shall be formalized in an Architectural Decision Record (ADR) detailing context, alternatives, and consequences.	Implemented for headline decisions (e.g., ADR-H5)
NFR-OPS-01	Operability	Backend services shall be containerized and independently deployable, with externalized configuration management.	Partial; backend containerized, root composition deferred
NFR-OPS-02	Operability	Backend services shall expose standardized health endpoints; readiness and metrics endpoints shall be provided where applicable.	Partial; /health implemented universally, /metrics on manager
NFR-OPS-03	Operability	The manager service shall emit structured telemetry including a correlation identifier; propagating structured logging across all services is a hardening direction.	Partial; manager only, cross-service propagation deferred
NFR-OPS-04	Operability	Synchronous operational endpoints shall maintain latency compatible with interactive user experiences under nominal load.	Observed during functional testing

3.4 Use Cases and Activity Diagrams

This section details the behavioral architecture of Itqan: the systemic use cases through which actors interact with the platform, and the activity flow defining the core lifecycle workflow.

3.4.1 System-Level Use Cases

Eight fundamental use cases define the interaction surface of Itqan. Table 3.4 presents the actor-use-case responsibility matrix, correlating actors with their authorized execution scopes.

Use Case	Manager	Estimator	Executive	Dept. User (future)	Admin (future)
Consult RFQ Portfolio	✓	✓ (scoped)	✓	✓	—
Register RFQ	✓	—	—	—	—
Advance RFQ Stage	✓	✓ (assigned)	—	✓ (own stage)	—
Manage Stage Artifacts	✓	✓ (assigned)	—	✓ (own stage)	—
Configure Reminders	✓	✓ (own)	—	✓ (own)	—
Consult Pipeline Analytics	✓	✓ (limited)	✓	—	—
Consult the RFQ Copilot	✓	✓ (scoped)	✓ (portfolio mode)	✓	—
Manage Workflow Templates	—	—	—	—	✓

Table 3.4: Actor-use-case responsibility matrix indicating access scope per role

The use cases are formally defined as follows. *Consult RFQ Portfolio* serves as the primary entry point, returning RFQs explicitly authorized by the actor’s permission scope. *Register RFQ* executes the intake of a new RFQ, associating identifying metadata with a specific workflow template (FR-MGR-01). *Advance RFQ Stage* mutates the RFQ lifecycle state, strictly gated by the validation criteria defined in FR-MGR-03. *Manage Stage Artifacts* handles the association of files, notes, and subtasks to a stage, enforcing append-only semantics for notes (FR-MGR-04). *Configure Reminders* orchestrates manual or automated rule-based reminders against stages or subtasks (FR-MGR-05). *Consult Pipeline Analytics* aggregates portfolio metrics, including status distributions, terminal win-rates, and client-specific breakdowns (FR-MGR-06). *Consult the RFQ Copilot* exposes a conversational interface to platform evidence via two contextual extension points: *portfolio mode* (global scope) and *RFQ-bound mode* (local scope) (FR-COP-01, FR-COP-02). *Manage Workflow Templates* governs the systemic templates driving stage instantiation and remains reserved for the future Administrator role.

3.4.2 Activity Diagram: Advance RFQ Stage

Among the defined use cases, *Advance RFQ Stage* optimally illustrates the deterministic governance implemented within the manager service. It represents the sole operation authorized to mutate the lifecycle state of a tracked RFQ, and execution is strictly blocked unless the three

validation gates of FR-MGR-03 are satisfied. The complex per-turn pipeline of *Consult the RFQ Copilot* is detailed separately in Chapter 4. Upon any gate validation failure, the workflow halts, and the persisted state remains immutably intact (NFR-REL-02). Conversely, when all gates validate successfully, the state mutation, timestamp generation, and parent-progress recomputation are committed to the database as a single, atomic transaction (NFR-REL-01), as illustrated in Figure 3.1.

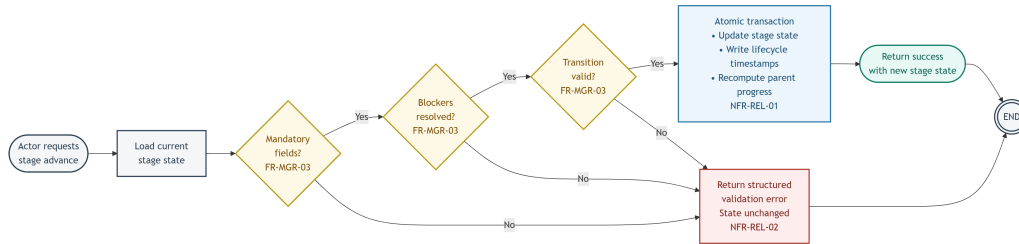


Figure 3.1: Activity diagram for *Advance RFQ Stage*

3.4.3 Conversational Flow with the Copilot

The conversational execution flow diverges structurally from the standard lifecycle workflows of the manager service: a single user interaction (turn) traverses a multi-stage pipeline, heavily relying on deterministic policy evaluations prior to any language-model invocation. This intricate pipeline architecture is detailed conceptually in Chapter 4 and technically in Chapter 5.

3.5 Methodology

The methodology governing this project was engineered to mitigate the constraints of a single-developer environment operating asynchronously with academic supervisors and a remote industrial client. The adopted frameworks ensure engineering rigor despite these constraints.

3.5.1 Scrumban as the Iteration Framework

Strict adherence to either Scrum or Kanban paradigms is suboptimal for a single-developer project characterized by asynchronous stakeholder availability. Traditional Scrum ceremonies lack efficacy without a dedicated team, whereas pure Kanban omits the structured demonstration cadence necessary for remote stakeholder alignment. Consequently, this project adopted *Scrumban*, a hybrid framework retaining Scrum's iterative cadence and demonstration discipline, while integrating Kanban's Work-In-Progress (WIP) limits to replace fixed-length sprints.

The Scrumban board implements a six-state workflow (Figure 3.2): *Backlog (Frozen Scope)*, *Ready*, *In Progress*, *Review and Demo*, *Done*, and *Documented*. Critically, the terminal *Documented* state integrates technical documentation directly into the Definition of Done (DoD). A feature transitions from *Done* only after its technical specification, architectural rationale, and validation evidence are formalized and committed synchronously with the codebase. This rigorous discipline ensures that this report is founded upon artifacts generated during active implementation rather than ex-post reconstruction.



Figure 3.2: Scrumban six-state workflow board, utilizing *Documented* as the terminal validation column

3.5.2 Architectural Decision Records

Architectural Decision Records (ADRs) adhere to a standardized six-part schema: context, options, decision, justification, consequences, and validation. ADR-H5 (*Dormant Models*) is committed to the manager repository as a fully formalized paradigm; subsequent decisions are systematically documented within Chapter 4.

3.5.3 Project Control Center

The Scrumban board, decision register, risk register, and stakeholder alignment matrix are centralized within a project Control Center deployed on Notion. This workspace is structured across twelve operational sections governing scope, backlog tracking, architectural decisions, and risk mitigation. The Control Center functioned as the single source of truth for project state

throughout the development iteration; data extracted from this centralized repository forms the empirical basis for the tables and artifacts presented in this chapter and throughout the report.

CONCLUSION

This chapter has formalized the systemic engineering structure of the project: identifying stakeholders, actors, and access models; delineating twenty-five functional and nineteen non-functional requirements; establishing behavioral use cases; and defining the agile methodology. The subsequent chapter formulates the technical architecture designed to satisfy these rigorous requirements.

Conceptual and Technical Architecture

Contents

INTRODUCTION	32
4.1 Global Architecture and Four-Pillar Layering	32
4.1.1 The Four Architectural Layers and the Cross-Cutting Copilot	32
4.1.2 The Four Pillars as an Architectural Map	33
4.2 Microservices Inventory and BACAB Pattern	34
4.2.1 Service Inventory	34
4.2.2 The BACAB Layered Pattern	35
4.2.3 Inter-Service Communication	36
4.3 Manager Service: Domain Model and Lifecycle	37
4.3.1 Mission and Domain Model	37
4.3.2 Workflow and Stage Progression	38
4.4 Copilot Service: Trust-Boundary Architecture	39
4.4.1 Design Philosophy	39
4.4.2 Positioning Against Alternative Conversational AI Patterns	40
4.4.3 The Eight Trust Boundaries	40
4.4.4 Two Classification Sources, One Plan Factory	42
4.4.5 The Path Registry as Policy-as-Configuration	43
4.4.6 The Pipeline and the Five Evidence Sources	43
4.4.7 Why This Architecture Represents the Project's Primary Innovation	43
4.5 Evidence Boundary	44
4.6 Platform Integration and ADR Summary	44
4.6.1 Platform Integration	44
4.6.2 Architectural Decisions	45
CONCLUSION	45

INTRODUCTION

This chapter formalizes the conceptual and technical architecture of the Itqan platform: the four-layer topological architecture and its corresponding four-pillar capability map (Section 4.1), the implementation of the BACAB layered software pattern (Section 4.2), the deterministic three-gate lifecycle of the manager service (Section 4.3), the strict trust-boundary architecture governing the LLM-powered copilot (Section 4.4), the formal evidence boundary constraints (Section 4.5), and the global platform integration alongside the architectural decision register (Section 4.6).

4.1 Global Architecture and Four-Pillar Layering

At the platform level, Itqan is topologically structured into four discrete layers and functionally mapped onto four capability pillars. The *layers* define the structural arrangement of systemic components, whereas the *pillars* define the functional capabilities realized by these structures. The RFQ Copilot intersects both paradigms as a cross-cutting conversational consumer, explicitly excluded from functioning as an independent structural layer or operational pillar.

4.1.1 The Four Architectural Layers and the Cross-Cutting Copilot

The **presentation layer** is encapsulated within the user interface service; the **orchestration layer** resides within the manager service, acting as the absolute authority for lifecycle representation and operational truth; the **intelligence layer** is governed by the intelligence service, which synthesizes deterministic analytical artifacts without owning core lifecycle state; and the **data layer** comprises the persistence infrastructure. Each persistence store is strictly private to its owning service, ensuring absolute encapsulation and boundary enforcement. The **RFQ Copilot** possesses no inherent authority over lifecycle, document, or analytical data; instead, it queries, interprets, and exposes evidence synthesized by the manager and

intelligence services. It traverses the presentation, orchestration, and intelligence layers purely as a read-oriented cross-cutting consumer. Its internal trust-boundary architecture is elaborated in Section 4.4. Figure 4.1 models the layered architecture and the copilot’s cross-cutting integration.

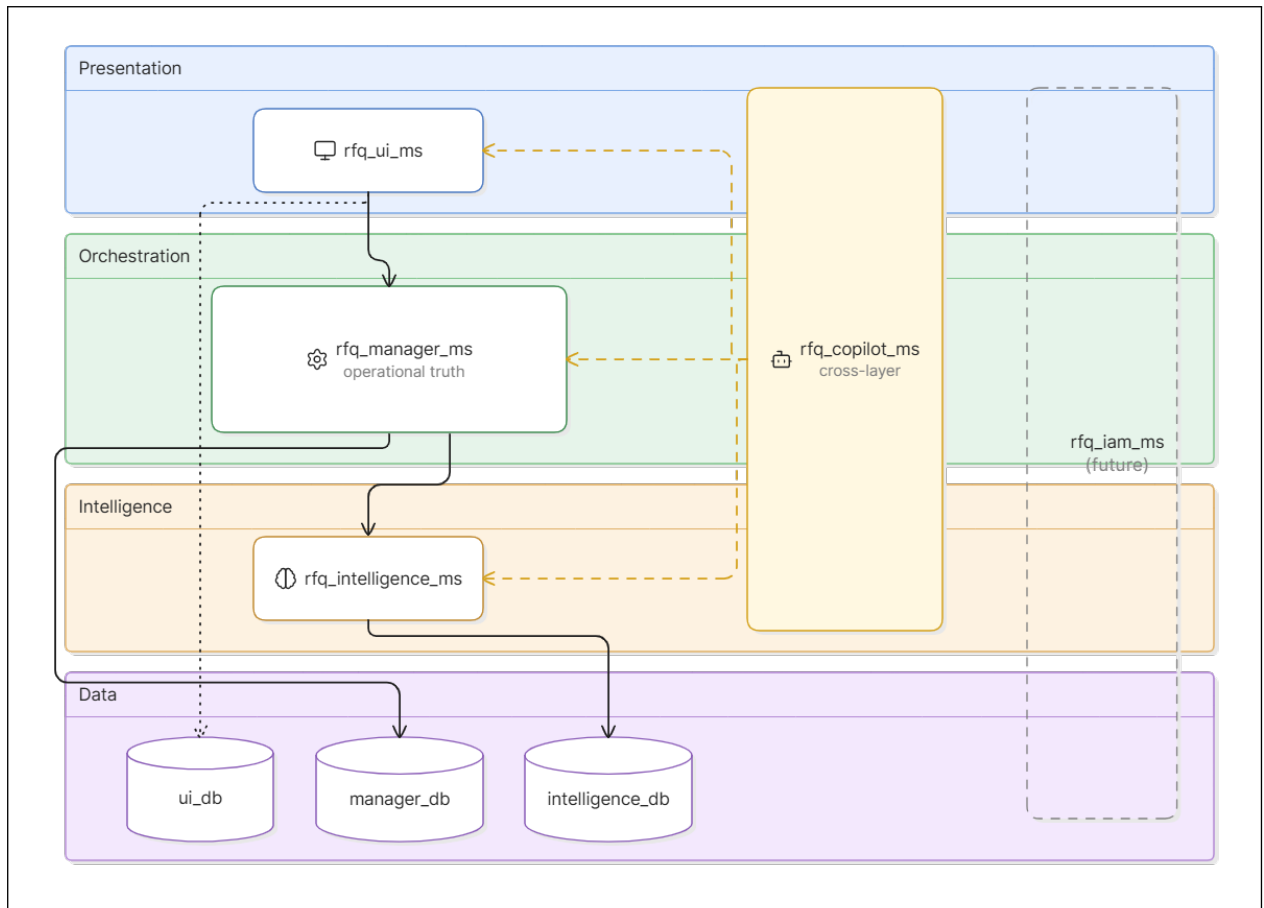


Figure 4.1: Global architecture of the Itqan platform: mapping of the four layers, microservices, and the cross-cutting RFQ Copilot

4.1.2 The Four Pillars as an Architectural Map

The four capability pillars introduced in Chapter 1 are mapped strictly to designated primary services: **Workflow and Tracking** is governed exclusively by the manager service; **Communication Automation** is partially realized via the manager service’s reminder infrastructure (with outbound multi-channel delivery designated as future scope); **Business Intelligence and Executive Visibility** is co-orchestrated by the manager service (data aggregation) and the user interface (dashboarding); **Historical Learning and Predictive**

Analytics is anchored in the intelligence service through deterministic document parsing, with advanced probabilistic modeling deferred. Table 4.1 details the operational status of each pillar.

Pillar	Status	Architectural Coverage
Workflow and Tracking	Implemented	Acts as the operational backbone of the platform, executed via the manager service across all primary lifecycle entities.
Communication Automation	Partial	Internal reminder infrastructure is deployed within the manager service; advanced outbound orchestration and inbound parsing remain future architectural extensions.
Executive Intelligence and BI	Partial	Real-time analytical endpoints are operational within the manager service; extensive BI reporting forms the future scope.
Historical Learning and Prediction	Foundation only	Deterministic parsing of Material Requisitions and estimation workbooks is operational within the intelligence service; predictive algorithms remain future scope.

Table 4.1: Operational scope of the four architectural pillars within the present project phase

4.2 Microservices Inventory and BACAB Pattern

4.2.1 Service Inventory

The distributed platform architecture is composed of **five microservices**: four are currently operational or partially implemented, while one is architecturally deferred. `rfq_manager_ms` constitutes the lifecycle orchestration backbone. `rfq_copilot_ms` encapsulates the trust-bounded conversational layer. `rfq_intelligence_ms` executes deterministic data

extraction over engineering documents. `rfq_ui_ms` serves as the presentation layer integrating live API consumption. `rfq_iam_ms` represents the deferred Identity and Access Management service; crucially, the manager service already embeds the requisite IAM integration point (Authorization-header forwarding and an identity-resolution connector, with token validation delegated to the future service). The microservice boundaries strictly mirror data ownership paradigms: the manager owns operational state, the intelligence service owns analytical models, the copilot encapsulates solely conversational state, and the UI owns presentation logic. This rigorous data encapsulation enables the trust-boundary architecture (Section 4.4): because the copilot holds no operational truth, its outputs must invariably rely on mediated evidence from authoritative backend services.

4.2.2 The BACAB Layered Pattern

Internally, each backend service adheres to the HostCompany architectural conventions, adapted into the **BACAB layered pattern**. This structure processes requests through five **active layers**: routes (HTTP ingress and syntactic validation), controllers (domain business logic), datasources (persistence abstraction), translators (Data Transfer Object (DTO) boundary transformations), and connectors (external service integration, including LLM APIs). These are transversally supported by three auxiliary layers: models, config, and utils. Three invariant architectural principles are strictly enforced: routing layers isolate business logic; controllers abstract database operations; and data access layers execute no domain decisions. Figure 4.2 illustrates this topology. The copilot service inherits this structural skeleton while injecting a specialized pipeline layer to orchestrate the trust-bounded LLM stages defined in Section 4.4.

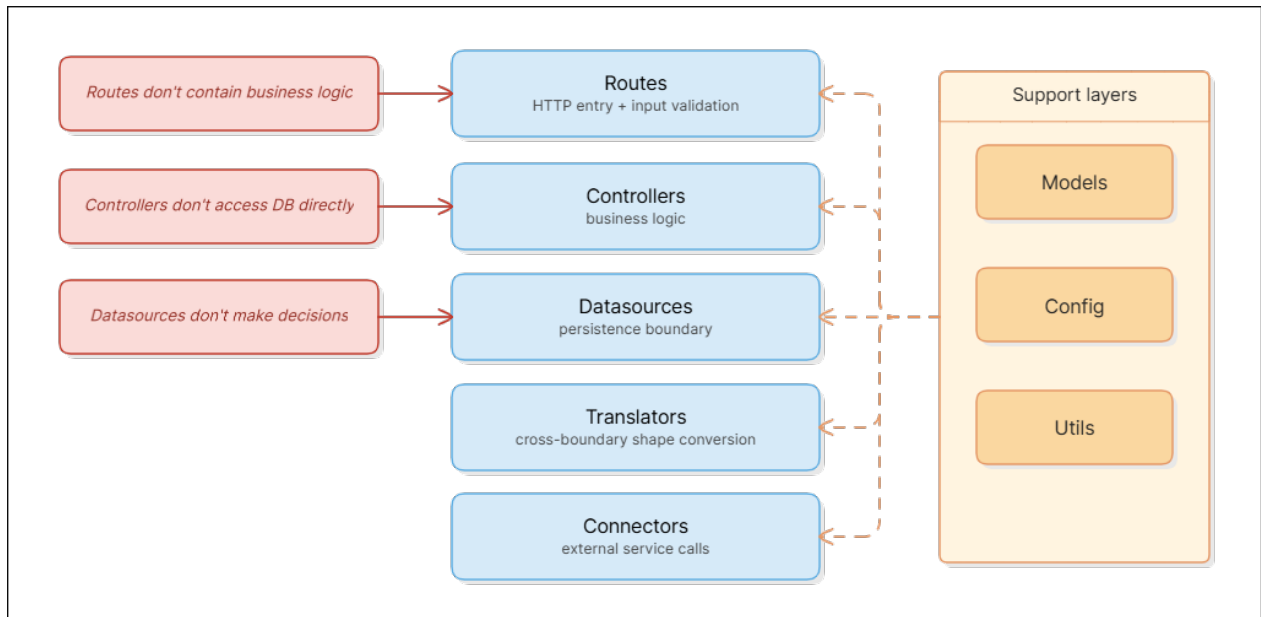


Figure 4.2: The BACAB layered architecture: structural separation of concerns via active and supporting layers

4.2.3 Inter-Service Communication

Inter-service communication is governed by three strict protocols. *Contracts:* Integration utilizes synchronous REST over HTTP, with each service exposing a versioned OpenAPI specification as the absolute source of truth. *Errors:* Exception payloads follow a standardized, structured format (HTTP status, machine-readable semantic type, and human-readable context). Access violations are explicitly categorized (401 Unauthenticated, 403 Unauthorized, 404 Resource Obfuscation for unauthorized actors). The manager service currently injects correlation identifiers, with holistic distributed tracing flagged as a hardening direction. *Access Control:* The copilot is strictly prohibited from bypassing manager-mediated authorization; all evidence retrieval traverses the manager's authenticated endpoints via explicit actor-context propagation, cementing the security premise of the trust-bounded architecture.

4.3 Manager Service: Domain Model and Lifecycle

4.3.1 Mission and Domain Model

The manager service operates as the absolute operational authority within the platform: it enforces a structured, deterministic lifecycle state machine for each RFQ. Its domain scope is deliberately bounded; the execution of cost estimation remains with human experts (Chapter 1), and complex analytical synthesis is delegated to the intelligence service. The persistence schema revolves around seven core entities (Figure 4.3): the **RFQ** (aggregate root, encapsulating metadata and state); the **Workflow** (versioned lifecycle template); the **RFQ_Stage** (instantiated operational node detailing transition requirements); the **Subtask** (granular execution unit supporting recursive progress aggregation); the **Note** (immutable append-only artifact); the **File** (document blob reference); and the **Reminder** (temporal automation trigger). Auxiliary mapping and configuration tables support this schema, as detailed in Chapter 5 (Section 5.2.2).

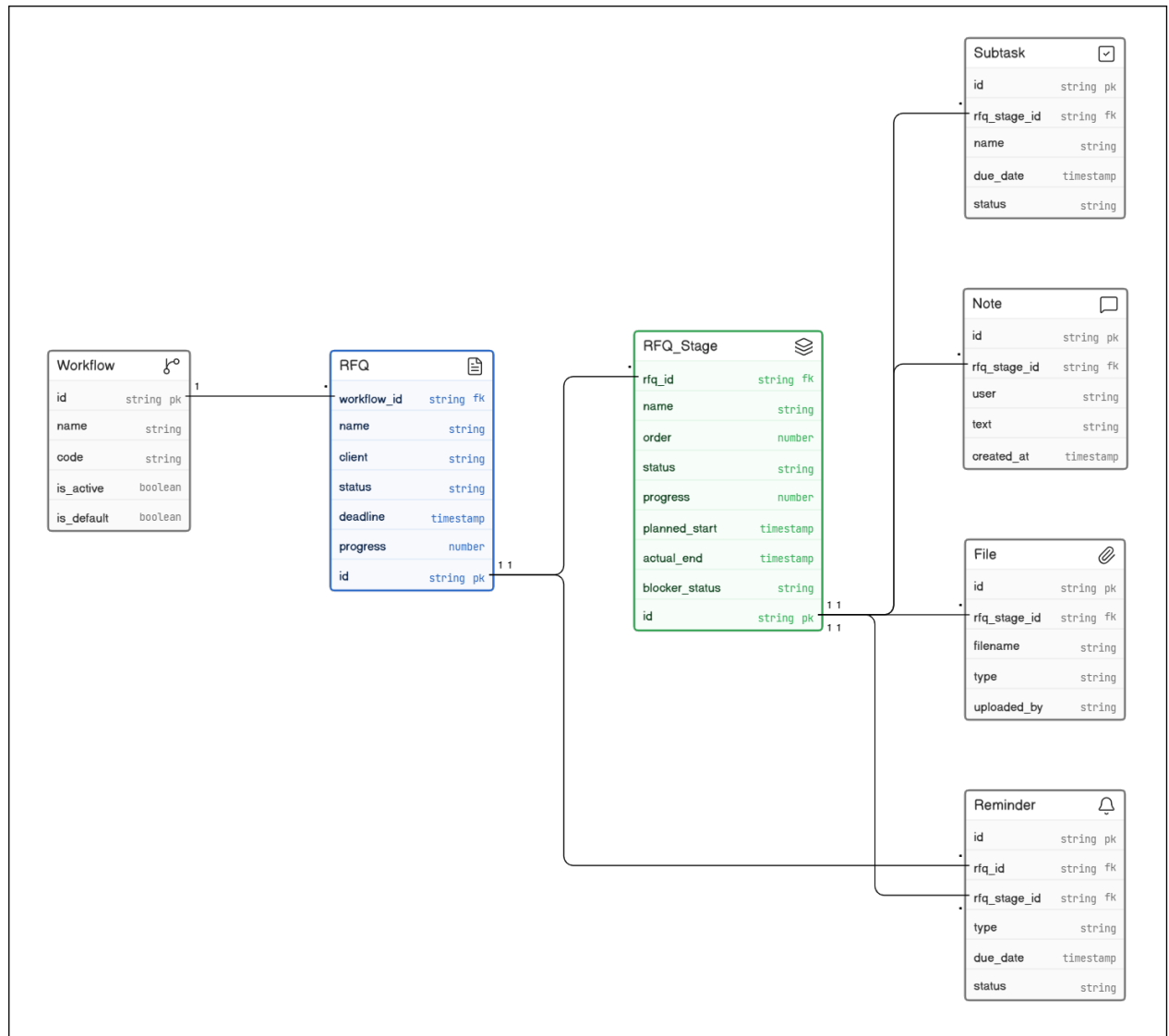


Figure 4.3: Entity-relationship diagram detailing the domain model of the manager service

4.3.2 Workflow and Stage Progression

The structural rigidity of the manager service is enforced via its deterministic stage progression logic. The mutation of an RFQ's state across lifecycle stages represents a highly guarded transactional transition, necessitating the successful traversal of three sequential validation gates, illustrated by the *Advance RFQ Stage* activity diagram (Figure 3.1). The *mandatory-field gate* aborts the transaction if predefined data dependencies for the current stage are unsatisfied. The *blocker gate* halts progression if any blocking constraint, such as an unresolved feasibility or coordination issue, remains active. The *transition-validity gate* enforces topological consistency, preventing arbitrary stage skipping or invalid sequencing by

validating that the requested transition concerns the current active stage and follows the ordered workflow sequence. Upon the successful validation of all gates, the state mutation commits atomically: the current stage is completed with an actual end timestamp, the next pre-created stage in the workflow sequence is activated with an actual start timestamp and an in-progress status, the RFQ's current-stage pointer is updated, and progress metrics are recomputed. The technical implementation of this transactional flow resides within the controller layer and is detailed in Chapter 5.

4.4 Copilot Service: Trust-Boundary Architecture

The RFQ Copilot operates explicitly as a constrained conversational interface rather than an autonomous agent. While the language model is relegated to semantic interpretation and text generation, deterministic algorithmic code strictly governs data access, tool execution, and the final authorization of responses.

This section formulates the underlying design philosophy, contrasts the copilot with alternative conversational paradigms, and defines the trust-boundary architecture that governs its pipeline logic. Implementation details are deferred to Chapter 5.

4.4.1 Design Philosophy

The architectural blueprint of the copilot is dictated by a singular, non-negotiable axiom:

The copilot must achieve conversational fluidity analogous to state-of-the-art LLMs, while explicitly abrogating their inherent generative authority. Its epistemic authority derives strictly from validated platform evidence, not from the probabilistic model.

This axiom decisively separates *fluidity* (the semantic capacity to parse intent and maintain linguistic coherence) from *authority* (the factual reliability required for operational decision-making). Generic conversational systems optimize for fluidity, inadvertently projecting an illusion of authority, a catastrophic vulnerability in industrial estimation contexts, where the cost of error is asymmetric (Chapter 1). Consequently, the architecture isolates these

capabilities: the language model generates fluid syntax, while deterministic microservice logic provisions and enforces absolute operational authority.

4.4.2 Positioning Against Alternative Conversational AI Patterns

The prevailing paradigms reviewed in Chapter 2 (naive RAG, autonomous LLM agents, and unconstrained function-calling architectures) exhibit a fundamental structural flaw when deployed in high-risk environments: they mistakenly grant the probabilistic language model autonomy over safety-critical decisions (e.g., context scope drift, tool selection, policy interpretation). The proposed trust-boundary architecture rectifies this by strictly reallocating responsibilities: semantic tasks (intent framing, response formulation, linguistic verification) are assigned to the LLM, whereas policy enforcement (tool authorization, evidence retrieval, logical escalation) is exclusively executed by deterministic algorithmic controls.

4.4.3 The Eight Trust Boundaries

The trust-boundary architecture isolates prompt classification from operational execution through a singular deterministic chokepoint. It systematically verifies generated responses via deterministic guardrails, culminating in a secondary LLM utilized strictly as a validating Judge. This architecture relies upon eight explicitly allocated boundaries (Table 4.2).

#	Decision Domain	Architectural Owner
1	Path Classification	<i>FastIntake</i> (deterministic regex) or <i>LLM Planner</i> (probabilistic proposal), structurally validated by the <i>PlannerValidator</i> and definitively authorized by the <i>ExecutionPlanFactory</i> .
2	Plan Construction Authority	Exclusively the <i>ExecutionPlanFactory</i> ; structural uniqueness is enforced via static anti-drift syntax-tree tests in the copilot's automated test suite.
3	Tool Invocation Policy	Declaratively mapped in the <i>Path Registry</i> and executed exclusively by the deterministic <i>Tool Executor</i> .
4	Data Authorization	Controlled entirely by the manager service API; HTTP 403 or 404 responses represent absolute barriers overriding copilot intent.
5	Prompt Context Boundaries	Enforced via a strict per-path field whitelist defined in the <i>Path Registry</i> .
6	Evidence Lineage Tracking	Per-target evidence labeling injected during the deterministic context-assembly phase.
7	Output Verification	Sequential <i>Deterministic Guardrails</i> followed by a terminal <i>LLM Judge</i> enforcing linguistic constraint validation.
8	Exception Escalation	The singular <i>Escalation Gate</i> , which deterministically halts the pipeline and re-routes execution via a structured exception framework.

Table 4.2: The eight deterministic trust boundaries defining the copilot's architecture

These boundaries are encapsulated in a unified architectural axiom: *Dual classification sources, a singular execution plan factory, and a unified escalation gate. The language model exclusively generates semantics; deterministic code constructs operational truth; declarative policy enforces boundaries; the LLM Judge validates outputs; and static templates guarantee safe degradation.* Any deviation from this axiom is categorically treated as a structural violation rather than an architectural evolution. Figure 4.4 maps the canonical pipeline realizing this logic.

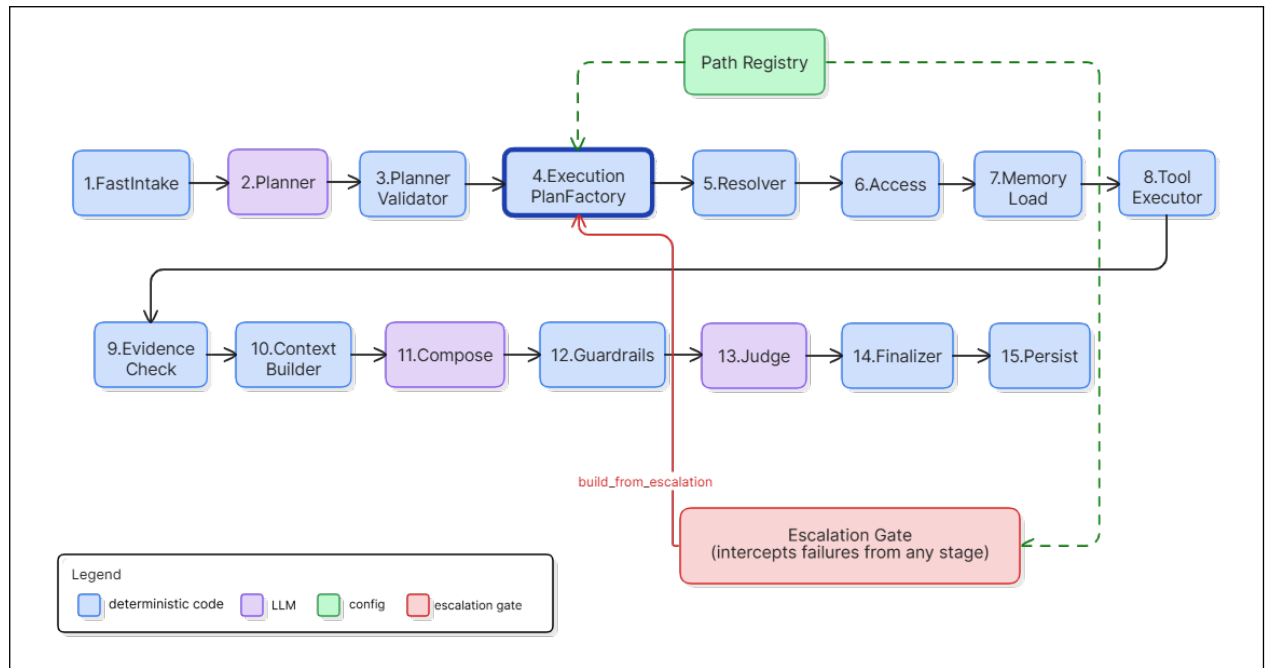


Figure 4.4: Canonical processing pipeline of the RFQ Copilot, demonstrating the strict encapsulation of LLM invocations within deterministic safeguards

4.4.4 Two Classification Sources, One Plan Factory

System interactions initiate exclusively via two distinct classification sources, both converging strictly onto a single point of execution plan construction. **FastIntake** serves as a deterministic classifier, identifying non-operational inputs (e.g., greetings, syntactic noise) utilizing anchored regular expressions, entirely bypassing the LLM. Conversely, complex intents are routed to the **LLM Planner**, which generates a probabilistic classification and data retrieval proposal. This proposal is intercepted by the **PlannerValidator**, which enforces strict schema conformance without executing policy logic. Both streams terminate at the **ExecutionPlanFactory**, the absolute single source of truth authorized to instantiate a `TurnExecutionPlan`. This singleton constraint is aggressively enforced via static anti-drift tests evaluating the project’s syntax tree. Any execution anomaly triggers the Escalation Gate, forcing a deterministic fallback (Path 8) response.

4.4.5 The Path Registry as Policy-as-Configuration

The Path Registry operationalizes the *policy-as-configuration* paradigm, defining the copilot's complete functional surface in a declarative matrix. It specifies permissible intake vectors, authorized retrieval tools, strict field whitelists, guardrail configurations, and targeted LLM Judge criteria for every intent path. Because policies are extracted from executable code into declarative configuration, altering systemic behavior constitutes a deterministic configuration update rather than a hazardous programmatic rewrite.

4.4.6 The Pipeline and the Five Evidence Sources

Execution traverses an immutable sequence: Intake → Factory → Resolver → Access → Memory → Tool Executor → Evidence Check → Context Builder → Compose → Guardrails → Judge → Finalize → Persist. The language model is explicitly bracketed at the Planner, Compose, and Judge nodes (Figure 4.4). The system recognizes five epistemic evidence sources: the **conversation context** (for trivial routing), **manager service state** (primary operational truth), **intelligence artifacts** (parsed engineering data), **controlled domain knowledge** (regulatory standards), and crucially, the **absence of evidence** (enforcing explicit refusal over probabilistic approximation).

4.4.7 Why This Architecture Represents the Project's Primary Innovation

The trust-boundary architecture transcends conventional conversational prototyping through rigorous structural and operational discipline. Structurally, it isolates the probabilistic LLM from execution authority via strict type definitions and self-defending static analysis (anti-drift tests). Operationally, every critical systemic decision is delegated to highly observable, deterministically auditable components (e.g., the Path Registry, the Tool Executor, the Guardrails). Academically, this architecture provides a replicable, engineered solution to

deploying LLM agents in environments where the asymmetrical cost of error precludes autonomous generative behavior.

4.5 Evidence Boundary

The *evidence boundary* represents the architectural invariant ensuring that the context supplied to the LLM is finite, explicitly authorized, and deterministically bounded prior to generative composition. At the *path level*, the ExecutionPlanFactory leverages the Path Registry to statically enforce which systemic data sources are permitted. At the *turn level*, the framework rigorously applies a per-path field whitelist and explicit evidence-lineage tagging to prevent context dilution and subsequent hallucination.

This boundary is rendered enforceable (rather than merely aspirational) via the declarative policy-as-configuration model, the singleton ExecutionPlanFactory constraint, and the sequential execution of deterministic guardrails followed by the LLM Judge. The Judge acts strictly as the terminal line of defense against semantic anomalies, never as the primary enforcer of access policy.

4.6 Platform Integration and ADR Summary

4.6.1 Platform Integration

The platform integration model dictates that no microservice may act as the secondary persistent owner of another service's domain data. The user interface integrates live REST endpoints from the manager (lifecycle orchestration) and intelligence (deterministic extraction) services, with a demo/mock fallback retained for development, while providing contextual access to the copilot interface.

4.6.2 Architectural Decisions

Critical architectural decisions (including the adoption of microservices, the BACAB intra-service topology, the centralization of lifecycle authority within the manager service, the policy-as-configuration framework, and the trust-bounded conversational paradigm) are documented within Architectural Decision Records (ADRs). These records detail the context, alternatives considered, chosen methodologies, and resulting operational consequences, as summarized in Appendix [A.10](#) (Table [A.6](#)). The trust-boundary architecture (Section [4.4](#)) represents the most consequential structural commitment, dictating the entire behavioral profile of the intelligence layer.

CONCLUSION

This chapter has formulated the comprehensive technical architecture of the Itqan platform: delineating the four-layer topological map, the five-service distributed framework, the intra-service BACAB pattern, and the deterministically guarded lifecycle within the manager service. Crucially, it established the trust-boundary architecture that safely encapsulates LLM capabilities within strict operational constraints, mitigating the severe risks of uncontrolled generative AI in industrial contexts. The subsequent chapter translates this conceptual architecture into precise programmatic implementation.

Implementation

Contents

INTRODUCTION	47
5.1 Development Environment and Toolchain	47
5.1.1 BACAB Discipline Across the Backend Services	49
5.2 Manager Service Implementation	49
5.2.1 BACAB Topology within the Manager Service	50
5.2.2 Relational Schema and Stage Progression	50
5.2.3 Subtasks, Notes, Files, and Reminders	51
5.2.4 Analytics and Telemetry Endpoints	52
5.2.5 Observability and Audit-History Implementation	52
5.3 Copilot Service Implementation	53
5.3.1 Turn Processing Pipeline and Vertical Slicing	53
5.3.2 Path Registry and Single-Construction Invariance	54
5.3.3 Memory, Guardrails, and the Escalation Gate	54
5.4 Intelligence Service Implementation	56
5.5 User Interface Integration	56
5.6 Cross-Service Integration Status	57
CONCLUSION	57

INTRODUCTION

This chapter details the programmatic implementation of the architecture formulated in Chapter 4. It examines the concrete realization of the BACAB layered pattern across the backend microservices, the deterministic lifecycle mechanics of the manager service, the per-turn pipeline orchestration of the copilot service, the parsing logic of the intelligence service, and the integration via the user interface. Implementation status is rigorously classified: operational and tested capabilities are documented as such, architecturally specified but unconnected interfaces are explicitly identified, and features relegated to future scope are strictly excluded from current achievements.

5.1 Development Environment and Toolchain

The three backend microservices are developed using Python 3.11 and the FastAPI 0.115 framework. SQLAlchemy 2 provides the Object-Relational Mapping (ORM) layer, while Pydantic 2 enforces strict boundary validation, with the translator layer mediating data transfer objects between them. The user interface is built as a single-page application utilizing TypeScript, Next.js 15, and React 18, styled via Tailwind CSS and the shadcn/ui component library.

The manager and intelligence services utilize PostgreSQL for relational persistence; the copilot employs SQLite for development-mode persistence of conversational threads, turn metadata, and execution records. The copilot's probabilistic language-model invocations interface with Azure OpenAI, with dynamic model selection governed by environment variables. Source code is versioned using Git within a polyrepo-in-monorepo structure. Containerization and Continuous Integration (CI) maturity vary across the platform, as codified in Table 5.1.

Architectural Layer	Technology Stack	Operational Role
Backend Services	Python 3.11 / FastAPI 0.115 / SQLAlchemy 2 / Pydantic 2	Core programming language, high-performance web framework, ORM engine, and boundary schema validation.
Frontend	Next.js 15 / React 18 / TypeScript / Tailwind CSS / shadcn/ui	Client-side application framework, strong typing, declarative styling, and component library.
Data Stores	PostgreSQL + SQLite	PostgreSQL for deterministic state persistence (manager, intelligence); SQLite for localized copilot session memory.
External Integrations	Azure OpenAI API	Probabilistic generative backend for the copilot, securely invoked via injected deployment variables.
Containerization	Docker (Backend focus)	Isolated container images for backend APIs; UI service and unified root composition remain in development.
Development Tooling	Git (Polyrepo-in-monorepo)	Distributed source control and repository orchestration.
Continuous Integration	Manager service exclusively	Automated CI workflow deployed for the manager; extension to copilot, intelligence, and UI is a prioritized hardening objective.

Table 5.1: Principal technological stack utilized for the implementation of the Itqan platform

Table 5.2 synthesizes the implementation evidence substantiating the delivery of the four primary platform components.

Component	Implemented Scope	Validation Evidence
rfq_manager_ms	RFQ lifecycle orchestration, workflow-derived pre-created staging, three-gate transactional advancement, granular subtasks, structured notes, file references, reminders, and analytical data endpoints.	259 executed tests (100% passing rate); validated OpenAPI contract; manual Postman integration checks.
rfq_copilot_ms	Trust-boundary pipeline architecture; operational Paths 1, 4, and the robust Path 8 safety fallback family; bounded working-memory capture; execution of guardrails and LLM Judge.	636 executed tests (100% passing rate); static anti-drift tests; formalized architecture specification.
rfq_intelligence_ms	Deterministic parsing foundation for Material Requisition (MR) packages and complex workbooks; heuristic anomaly detection; briefing artifact foundation and briefing assembly in progress.	118 collected tests; 64 passing, 48 fixture-dependent (golden artifacts excluded from Git), 6 skipped.
rfq_ui_ms	Portfolio visualization, RFQ detail interface, copilot conversational drawer, dashboard analytics, and intelligence artifact panels.	12 source-contract validation scripts (11 passing); UI demonstrability within the current prototype scope.

Table 5.2: Implementation evidence synthesis across delivered platform components (test counts derived from the project snapshot detailed in Appendix A.4)

5.1.1 BACAB Discipline Across the Backend Services

All three backend microservices rigorously implement the BACAB layered pattern (Chapter 4, Section 4.2) as their foundational software architecture. The manager service is detailed subsequently as the most mature instantiation; the intelligence service strictly mirrors this topology; and the copilot service inherits the BACAB skeleton while injecting a dedicated pipeline module to orchestrate the per-turn stages defined in Chapter 4.

5.2 Manager Service Implementation

The manager service functions as the operational hub of the platform. Its implementation encompasses RFQ registration, workflow-driven lifecycle staging, transactional stage progression, hierarchical subtasks, immutable notes, file telemetry, reminder execution, audit-history scaffolding, and analytical aggregation.

5.2.1 BACAB Topology within the Manager Service

The manager service operationalizes the BACAB pattern via a strict directory taxonomy that reflects the active-and-support layer dichotomy. A specific programmatic deviation consolidates Pydantic request/response schemas alongside translator mapping functions within the translators layer, rather than isolating them. Each Python module maps exactly to a designated architectural layer. The three invariant discipline rules (Chapter 4, Section 4.2) are absolute, enforced structurally by linting constraints and CI workflow policies. Listing 5.1 illustrates this topological layout.

```
rfq_manager_ms/  
+-- src/  
|   +-- controllers/      # Domain orchestration logic (6 modules)  
|   +-- routes/           # HTTP ingress and validation (6 modules)  
|   +-- datasources/      # ORM abstraction and queries (6 modules)  
|   +-- models/           # SQLAlchemy relational entities (12 tables)  
|   +-- translators/      # DTO schemas and mapping logic  
|   +-- connectors/       # External system integration (event_bus, IAM)  
|   +-- services/         # Cross-cutting operational logic  
|   +-- config/, utils/  
+-- migrations/           # Alembic schema versioning  
+-- tests/                # Pytest automated suite (259 passing tests)
```

Listing 5.1: Topological BACAB layout of rfq_manager_ms (depth 2)

5.2.2 Relational Schema and Stage Progression

The persistence layer comprises twelve relational tables hosted in PostgreSQL. The seven primary entities of the domain model (RFQ, Workflow, RFQ_Stage, Subtask, Note, File, Reminder) translate symmetrically (1:1) into SQLAlchemy ORM models. Five auxiliary tables support complex state management: stage_template, rfq_stage_field_value, reminder_rule, rfq_code_counter, and the reserved rfq_history audit table.

Lifecycle progression synthesizes the three theoretical gates (mandatory-field, blocker, transition-validity) into a singular, ACID-compliant database transaction. Each gate evaluates distinct criteria; failure at any gate halts the process, yielding a structured exception detailing the exact constraint violation. Upon unanimous validation, the transaction commits atomically: the current stage is marked as completed with an actual end timestamp, the next pre-created stage in the workflow sequence is activated, and completion metrics are recalculated. This guarantees that the RFQ lifecycle never enters an inconsistent or partial state. Lifecycle traceability relies inherently on chronological timestamps, persisted stage states, append-only constraints on notes, and non-destructive entity updates. Listing 5.2 demonstrates the HTTP route interface executing this transactional progression.

```
# routes/rfq_stage_route.py (representative excerpt)
@router.post("/rfqs/{rfq_id}/stages/{stage_id}/advance")
def advance_stage(rfq_id: UUID, stage_id: UUID, ...):
    # Controller orchestrates a singular ACID transaction:
    # 1. Mandatory-field gate: Validates completeness
    #    of required inputs.
    # 2. Blocker gate: Confirms resolution of all
    #    blocking constraints.
    # 3. Transition-validity gate: Verifies target
    #    against canonical workflow.
    # Upon unanimous success: completes current stage,
    # activates the next pre-created stage, updates metrics.
    # Upon failure: rolls back transaction,
    # returns structured constraint exception.
    return controller.advance(rfq_id, stage_id, ...)
```

Listing 5.2: Representative HTTP endpoint executing the three-gate stage advancement transaction. (Note: Dependency injection and error handling obfuscated for brevity).

5.2.3 Subtasks, Notes, Files, and Reminders

Four subsidiary resource models complement the core lifecycle architecture. **Subtasks** function as granular execution increments attached to specific stages, implementing soft-delete

semantics: deprecated subtasks are timestamped and retained, while being excluded from recursive progress calculations. **Notes** enforce strict append-only constraints: the API exposes solely POST and GET methods, structurally prohibiting PUT and DELETE operations to preserve the integrity of the communication audit trail. **Files** represent binary object references, explicitly categorized by MIME type; this includes a designated estimation workbook type primed for deterministic parsing by the intelligence service (autonomous event-driven consumption is architecturally prepared but currently inactive). **Reminders** constitute temporal execution triggers, supporting both manual instantiation and rule-based generation.

5.2.4 Analytics and Telemetry Endpoints

The manager service provides a robust analytical interface, aggregating operational RFQ data via two primary REST endpoints: `/rfqs/stats` computes high-level portfolio KPIs (aggregate volume, active queue size, critical path flags, and mean cycle time), while `/rfqs/analytics` delivers multi-dimensional win-rate breakdowns (terminal status categorization aggregated by client or workflow template). Advanced financial analytics (margin analysis and estimation accuracy) require longitudinal outcome data currently outside the system's capture parameters, and are consequently deferred to future development phases.

5.2.5 Observability and Audit-History Implementation

The manager service is packaged via a Dockerfile and orchestrated through a scenario-specific docker-compose configuration. The service emits structured telemetry, injecting correlation identifiers into request headers to facilitate distributed tracing across routes, controllers, and datasources. Enhancing cross-service identifier propagation remains a critical observability objective. System health is monitored via standard `/health` liveness probes and a `/metrics` endpoint instrumented via the official Prometheus client.

Crucially, the database schema implements an `rfq_history` table designed to capture generic, immutable audit logs across all lifecycle entities. However, no active write paths currently populate this table (a deliberate architectural constraint formalized in Architectural

Decision Record H5). Lifecycle traceability currently relies on implicit temporal state variables; the activation of explicit audit logging is flagged as a high-priority hardening requirement.

5.3 Copilot Service Implementation

The copilot codebase is engineered across two parallel release lanes: a **v1 lane** that established the foundational conversational interfaces, and a **v2 lane** that rigorously implements the deterministic trust-boundary architecture (Chapter 4). The most recent operational slice finalized v2 thread management and bounded working-memory capture. Consequently, the v2 trust-boundary pipeline is the execution path for every conversational turn, while the v1 thread-management routes remain mounted and are still consumed by the frontend (the thread-lifecycle cutover remains a pending alignment task).

Structurally, v2 leverages the BACAB paradigm, isolating a distinct pipeline layer between the controllers and data access objects. This layer encapsulates every discrete execution stage (Intake, Planner, Validator, Factory, Resolver, Access, Tool Executor, Evidence Check, Context Builder, Compose, Guardrails, Judge, Escalation Gate, Finalizer, and Persist) as independent Python modules, orchestrating them sequentially via the primary turn controller.

5.3.1 Turn Processing Pipeline and Vertical Slicing

A user query initiates a strictly linear sequence of execution stages. Each stage is responsible for a singular decision matrix: **FastIntake** → **LLM Planner** → **PlannerValidator** → **ExecutionPlanFactory** → **Resolver** → **Access (manager-mediated authorization)** → **Tool Executor** → **Evidence Check** → **Context Builder** → **Compose** → **Guardrails** → **Judge** → **Finalizer** → **Persist**. Working memory is loaded into the execution state by the turn controller after plan construction, explicitly preserving the ExecutionPlanFactory's isolation; in the current slice it is not injected into the Planner, Compose, or Judge prompts. This sequence corresponds to the canonical pipeline conceptualized in Chapter 4 (Figure 4.4), which distinguishes deterministic programmatic execution from probabilistic LLM invocations.

The initial vertical slice validates three critical execution paths end-to-end: **Path 1** (Deterministic intake routing for trivial conversational inputs), **Path 4** (The core operational capability executing evidence-grounded RFQ queries via authorized manager connectors), and the robust **Path 8** safety matrix handling five distinct failure modes. Capabilities designated as Paths 2, 3, 5, 6, and 7 are architecturally defined but systematically routed to Path 8.1 (Unsupported Operation) via the Escalation Gate pending their respective implementations.

5.3.2 Path Registry and Single-Construction Invariance

The **Path Registry** functions as the declarative policy matrix, parsed at runtime exclusively by the **ExecutionPlanFactory** and the Escalation Gate. The **Planner** leverages a structured Azure OpenAI invocation, constrained by JSON schema, to probabilistically classify intent. The **PlannerValidator** enforces structural and schema integrity on this output, operating strictly without referencing the policy registry.

The **ExecutionPlanFactory** serves as the absolute architectural chokepoint, uniquely authorized to instantiate a **TurnExecutionPlan**. Three fundamental architectural invariants are aggressively enforced via static syntax-tree (anti-drift) tests within the copilot's automated test suite: (1) *Single-Construction Anomaly Detection*, ensuring no external code instantiates the execution plan; (2) *Registry-Reader Whitelisting*, restricting Path Registry access exclusively to authorized components; (3) *Deterministic Intake Enforcement*, demanding that **FastIntake** executes solely via anchored regex evaluations devoid of network or LLM latency.

5.3.3 Memory, Guardrails, and the Escalation Gate

The working memory sub-system currently operates in a capture-only paradigm: contextual turn data is persisted but not recursively injected into the generative prompt (an isolation enforced by anti-drift testing). The **Guardrails** layer executes deterministic validation on the generated response (validating evidence grounding and schema conformity). The **Judge** component initiates a secondary, constrained LLM invocation to classify the generated text against severe anomalies (fabrication, unauthorized inference).

The **Escalation Gate** acts as the universal deterministic intercept mechanism for pipeline failures. It algorithmically maps any exception (e.g., target obfuscation, evidence sparsity, guardrail failure) to a corresponding Path 8 protocol, dynamically re-routing execution back through the factory to synthesize a safe, structured degradation response rather than throwing a raw system exception. Crucially, the Gate never constructs a plan itself: it raises a structured `EscalationRequest` and re-enters `ExecutionPlanFactory.build_from_escalation()`, preserving the single-construction invariant even on the failure path. The trigger determines the target sub-case among the five members of the Path 8 family (8.1 unsupported/invalid, 8.2 out-of-scope, 8.3 clarification, 8.4 inaccessible, 8.5 no-evidence/service-unavailable), while the `reason_code` selects the specific safe-response template rendered by the Finalizer (Figure 5.1).

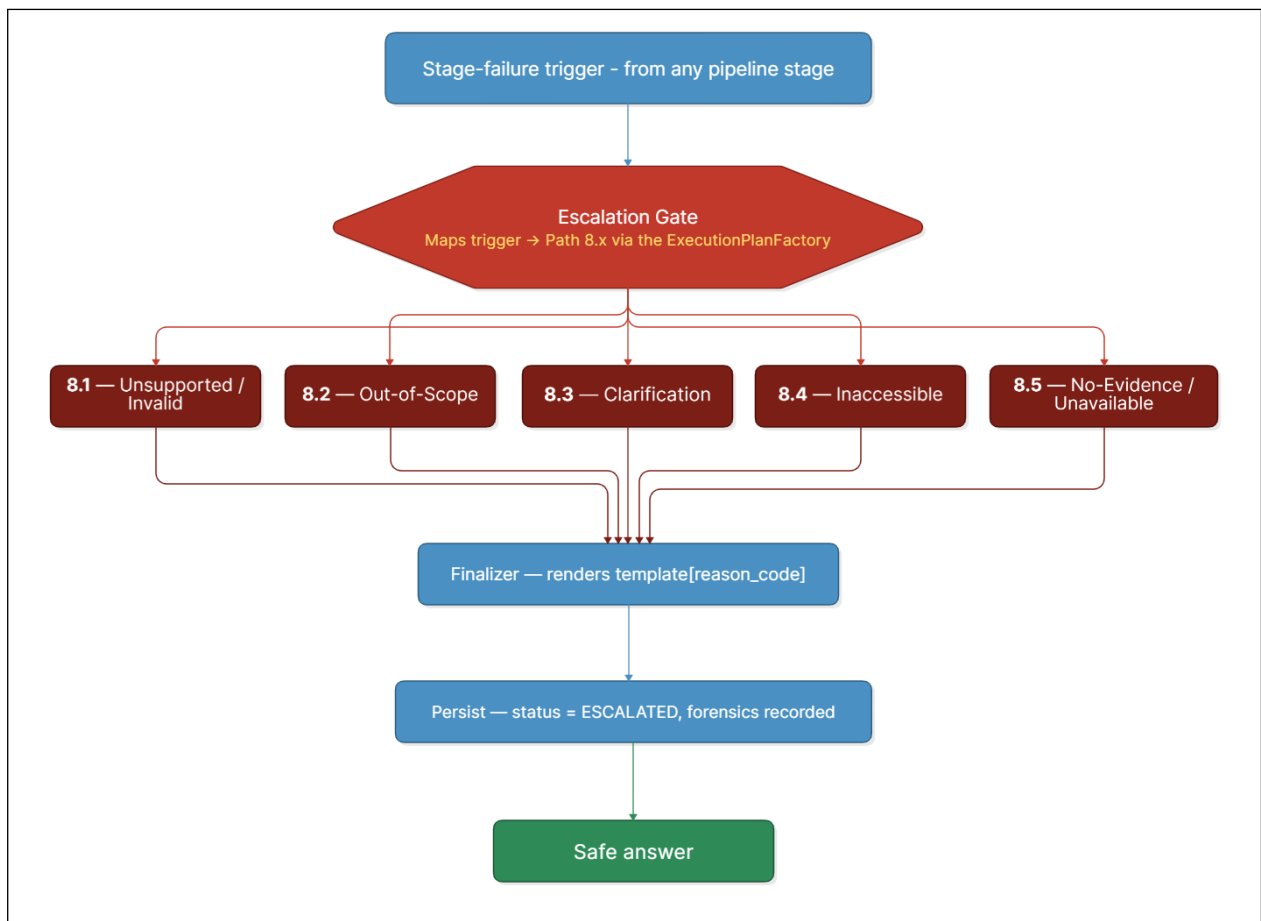


Figure 5.1: The Escalation Gate and the Path 8 safety family: any stage-failure trigger is deterministically mapped, via a single re-entry into the `ExecutionPlanFactory`, to one of five Path 8.x safe-degradation responses, with the reason code selecting the rendered template

5.4 Intelligence Service Implementation

The intelligence service is implemented as a specialized FastAPI module adhering to the identical BACAB architecture, organizing extracted artifacts through a PostgreSQL-backed artifact model. The data model is structurally unified: a central artifact table stores polymorphic entities including MR package profiles, analytical workbook parameters, and automated briefings, indexed by RFQ identifier.

The **MR package parser** deterministically maps canonical document structures (e.g., benchmark SA-AYPP-6-MR-022) to extract material taxonomies, vendor constraints, and technical metadata. The **workbook parser** anchors extraction to a three-sheet signature set (*General*, *Bid S*, *Top Sheet*) within the 32-sheet GHI reference workbook (e.g., fixture IF-25144), extracting cost-related values and schedule-relevant parameters where supported by the workbook structure. Critically, anomalous or inconsistent variables are flagged through deterministic and heuristic checks, avoiding silent acceptance of corrupted data.

5.5 User Interface Integration

The presentation layer (`rfq_ui_ms`) is a Next.js single-page application utilizing TypeScript and React, designed for live backend consumption while retaining mock/demo fallbacks during prototype hardening. The interface provides centralized portfolio visualization, granular RFQ detail metrics (lifecycle state, hierarchical subtasks, document telemetry), and integrated intelligence artifact rendering. The conversational copilot interacts via a dynamic UI drawer instantiated within the RFQ context.

As a prototype interface, several architectural parameters remain partial: authentication relies on client-side state mapping rather than robust OIDC flows; UI containerization and comprehensive E2E browser automation (e.g., Cypress) are currently omitted; and the copilot frontend layer momentarily leverages legacy v1 routing, awaiting alignment with the v2 execution backend. Figure 5.2 illustrates the primary interactive dashboard.

[Figure 5.2 placeholder: representative screen of the user interface, showing the RFQ detail view with the lifecycle timeline, the intelligence panel, and the copilot drawer in RFQ-bound mode.]

Figure 5.2: Representative UI dashboard interface demonstrating the integration of the lifecycle orchestration panel and the copilot conversational drawer

5.6 Cross-Service Integration Status

Platform integration paradigms are comprehensively validated in Chapter 6. The **manager-to-intelligence pathway** operates via direct REST triggers, whereas the **copilot-to-evidence pathway** securely accesses data strictly through the manager service's authenticated endpoints. The direct copilot-to-intelligence integration remains architecturally defined but programmatically deferred (Path 5).

CONCLUSION

This chapter thoroughly documented the software implementation of the Itqan architecture. It detailed the stringent enforcement of the BACAB pattern across microservices, the transactional integrity of the manager service's relational schema, the programmatic realization of the copilot's trust-boundary pipeline via single-construction design, and the algorithmic parsing strategies of the intelligence layer. The ensuing chapter provides the empirical validation and verification evidence substantiating these implemented capabilities.

Validation, Results, and Discussion

Contents

INTRODUCTION	59
6.1 Validation Strategy and Five Evidence Classes	59
6.2 Manager Service Validation	60
6.3 Copilot Service Validation	61
6.4 Trust-Boundary Verification via Anti-Drift Constraints	62
6.5 Intelligence Service Validation	63
6.6 Frontend and Cross-Service Integration Validation	63
6.7 Business Impact Mapping	64
6.8 Limitations and Future Work	64
CONCLUSION	65

INTRODUCTION

This chapter structurally validates the Itqan platform against the rigorous engineering requirements formulated in Chapter 3 and the architectural constraints defined in Chapter 4. It presents empirical evidence confirming the platform’s deterministic behavior, evaluates the business impact relative to the operational vulnerabilities identified in Chapter 1, and concludes with a critical discussion of the current system’s limitations and projected architectural hardening.

6.1 Validation Strategy and Five Evidence Classes

The validation methodology is anchored in five complementary classes of empirical evidence: **service-level automated testing** (pytest suites for backend microservices), **architectural invariant verification** (static anti-drift tests enforcing the trust-boundary paradigm), **scenario-level execution** (integration sequences exercising the safety matrices), **integration evidence** (manual and scripted cross-service API validation), and **business-impact analysis** (a deterministic mapping of realized capabilities to the primary operational constraints).

This methodology acknowledges explicit boundary limits (e.g., CI execution restricted to the manager service, SQLite as the current quality-gate target, reliance on deterministic LLM mock interfaces for automated testing, and the absence of browser-level E2E automation). These limits are rigorously consolidated in Section 6.8. Table 6.1 aggregates the validation evidence per architectural domain.

Domain	Evidence Scope	Outcome	Methodological Constraint
Manager Service	259 automated pytest tests	100% Success	Validated against an SQLite target; explicit Postman network validations pending.
Copilot Service	636 automated pytest tests, supplemented by structured manual integration testing	100% Success	Automated suites leverage deterministic LLM test doubles; live-model regression automation remains future work.
Intelligence Service	118 collected pytest tests	64 Success, 48 fixture-dependent, 6 skipped	Fixture-dependent suites require proprietary data assets omitted from the audited repository.
Frontend UI	12 source-contract validation scripts	11 Success, 1 assertion drift	Complete absence of browser-level end-to-end (E2E) automation in the current iteration.
Integration	Sequential scenario scripts, structured data seed manifests, and manual API checks	Validated within the current manual/scripted scope	Absence of fully automated, unassisted full-stack E2E execution runs.

Table 6.1: Consolidated aggregation of empirical validation evidence

6.2 Manager Service Validation

Validation of the manager service (Table 6.2) leverages a suite of **259 tests**, achieving a 100% pass rate against the SQLite quality gate (Appendix A.4). The suite exercises the seven core entity families (FR-MGR-01 to 08), targeting nominal success, systematic validation failure, resource obfuscation (HTTP 404), and actor-context and access-control scenarios.

The three-gate transactional advancement mechanism (Chapter 5) is validated across both acceptance and rejection matrices. Transactional atomicity is verified via intermediate-failure simulation, with the recognized concurrency limitation (LG-06) logged for subsequent mitigation. Consistency testing validates soft-delete paradigms (FR-MGR-08), structured note persistence, exact MIME-type file discrimination, and reminder orchestration. The declarative audit-history schema remains latent, categorized as a critical hardening objective in Section 6.8.

Validation Vector	Status	Architectural Notes
Service-level Pytest Suite	259 Passed (0 Failed)	Comprehensive FR-MGR-01 to 08 coverage; execution restricted to SQLite.
Transactional Stage Gates	Implemented & Verified	Validates mandatory-field, blocker, and structural sequence gates; acknowledges LG-06 concurrency limits.
Secondary Resource Semantics	Implemented & Verified	Enforces soft-delete, append-only integrity, file taxonomies, and reminder triggers.
Systemic Hardening	Pending Action	Integration of PostgreSQL testing, Newman telemetry, and audit-history activation.

Table 6.2: Validation evidence matrix for the manager service

6.3 Copilot Service Validation

The copilot testing suite executes **636 tests** within a few seconds (Appendix A.4). It is architected across three rigorous categories (Table 6.3): **pipeline tests** (216 functions validating isolated stage logic), **integration smoke tests** (98 functions executing end-to-end traversal), and **anti-drift tests** (18 functions statically enforcing the trust-boundary invariants, elaborated in Section 6.4). The remaining 304 tests target data access objects, external contracts, configuration matrix bounds, and controller isolation. The probabilistic stages (Planner, Compose, Judge) are deterministically stubbed via a FakeLlmConnector; live Azure OpenAI evaluation is identified as a critical future validation metric.

Test Taxonomy	Count	Verification Scope
Pipeline Verification	216	Isolated execution validation of FastIntake, Planner, Validator, Factory, Resolver, Access, Tool Executor, Evidence Check, Context Builder, Compose, Guardrails, Judge, Finalizer, and the Escalation Gate.
Integration Scenarios	98	End-to-end traversal: noise rejection, RFQ-grounded data retrieval, structured fallback logic, access denial enforcement, and operational persistence.
Anti-Drift Guardrails	18	Static syntax-tree analysis enforcing architectural invariability (detailed in Section 6.4).
Ancillary Validations	304	Validation of DAOs, data models, configuration schemas, API contracts, and controller routing.
Total Coverage	636	100% Pass Rate

Table 6.3: Taxonomic breakdown of the copilot service automated test suite

6.4 Trust-Boundary Verification via Anti-Drift Constraints

The eight deterministic trust boundaries (Chapter 4, Section 4.4) are strictly enforced through a synthesis of **anti-drift tests** (static syntax-tree analysis preventing structural regression, executed with the copilot test suite) and **pipeline tests** (dynamic checks evaluating runtime behavior). Anti-drift tests guarantee structural integrity prior to runtime execution, whereas pipeline tests intercept behavioral deviations. Table 6.4 delineates this enforcement mapping.

#	Trust Boundary (Section 4.4)	Enforcement Layer	Verification Mechanism
1	Path Classification Authority	Anti-Drift	FastIntake regex anchoring; absolute enforcement of Planner output schemas.
2	Singular Plan Construction	Anti-Drift	Static verification of <code>TurnExecutionPlan</code> single-factory instantiation.
3	Tool Invocation Policy	Anti-Drift	Execution constrained strictly via the Path Registry whitelist.
4	Data Authorization	Pipeline	Runtime assertion of manager-mediated Access stage routing.
5	Prompt Context Isolation	Anti-Drift	Enforcement of strict per-path field whitelisting prior to LLM submission.
6	Evidence Lineage Tracking	Pipeline	Runtime assertion of per-target lineage tagging mechanisms.
7	Output Verification	Anti-Drift	Prohibits SDK logic in deterministic layers; enforces memory-policy integrity.
8	Exception Escalation Protocol	Pipeline	Runtime validation of Escalation Gate routing logic.

Table 6.4: Methodological mapping of trust boundaries to their specific enforcement layers

The 18 anti-drift tests monitor boundaries 1, 2, 3, 5, and 7 (those vulnerable to static structural degradation). Boundaries 4, 6, and 8, which depend on operational latency and external inputs, are validated dynamically. This dual-layered verification cements the single-construction discipline of the `ExecutionPlanFactory`, transforming theoretical architectural constraints into verifiable software invariants.

6.5 Intelligence Service Validation

The intelligence service validation suite collects **118 tests**. Currently, 64 fixture-independent tests execute successfully, 6 are intentionally bypassed, and 48 depend exclusively on proprietary external fixtures (the canonical MR package and estimation workbooks). These proprietary files trigger a `FileNotFoundException` when unavailable in the local environment, accurately reflecting a data availability constraint rather than a demonstrated computational parser regression. Core fixture-independent logic (including briefing assembly, API intake, and telemetry) executes successfully; end-to-end parser validation against the canonical MR package and estimation workbooks depends on the proprietary fixtures exercised by the 48 fixture-dependent tests. The integration of this service is verified within the current manual and scripted scope, as detailed in Appendix [A.7](#) (Table [A.3](#)).

6.6 Frontend and Cross-Service Integration Validation

The platform's operational coherence relies on seven defined cross-service integration vectors (detailed in Appendix [A.7](#), Table [A.3](#)). The presentation layer relies on twelve structured Node source-contract scripts, eleven of which pass; the sole anomaly derives from a localized assertion drift rather than an integration failure. These scripts inspect connector and source contracts and do not constitute browser-level end-to-end validation; advanced DOM-level browser testing (e.g., via Playwright) remains outside the current operational perimeter.

Cross-service communication leverages strict OpenAPI protocols: UI to Manager (lifecycle rendering); UI to Copilot (conversational turn management); Copilot to Manager (authenticated evidence retrieval); Manager to Intelligence (deterministic triggers). The integration between the Copilot and the Intelligence service is architecturally mapped (Path 5) but programmatically deferred. Full integration hardening relies heavily on structured scenario matrices and realistic seed manifests.

6.7 Business Impact Mapping

Empirical validation supports the system’s alignment with its defined architectural matrix. This section correlates these implemented capabilities directly to the operational vulnerabilities (PP-01 through PP-12) delineated in Chapter 1. Each constraint is classified, based on the implemented scope, as *Addressed*, *Partially Addressed*, or *Not Yet Addressed*.

Resolution Status	Identified Constraints	Architectural Resolution Mechanism
Addressed (2)	PP-01, PP-08	Realized via a unified lifecycle taxonomy, pre-created workflow stages, and the transactional transition-validity gate.
Partially Addressed (7)	PP-02, PP-03, PP-05, PP-06, PP-07, PP-11, PP-12	Advanced through reminder infrastructure, quantitative analytics endpoints, deterministic workbook parsing foundations, structured notes, file metadata, and anomaly flagging. Complete resolution requires advanced margin analytics and outbound automation.
Not Yet Addressed (3)	PP-04, PP-09, PP-10	Predictive supplier risk modules (Pillar 4) and ERP integration vectors remain outside the current implementation scope.

Table 6.5: Synthesis of business impact mapping against initial operational constraints

The platform yields a deterministic **2/7/3 resolution matrix**. This distribution reflects the strategic project pivot (detailed in Chapter 1) away from localized automation toward systemic lifecycle intelligence, showing that the foundational infrastructure is in place while clearly bounding future capabilities.

6.8 Limitations and Future Work

Every engineered system operates within explicit structural boundaries. This section catalogues the platform’s current operational constraints and outlines the immediate trajectory for architectural mitigation (Table 6.6).

Constraint Cluster	Observed Limitations	Strategic Mitigation Trajectory
Validation & Testing Gaps	CI automation restricted to manager service; testing executed against SQLite rather than PostgreSQL; fixture-dependent parser gaps; absence of a curated evaluation benchmark for LLM output quality.	Deploy unified CI/CD pipelines across all microservices; migrate test targets to PostgreSQL; formalize reproducible testing fixtures; construct a rigorous quantitative benchmark for probabilistic LLM evaluation.
Architectural Deferrals	Dedicated IAM microservice pending; working memory persists but avoids recursive prompt injection; Path 5 (Intelligence Connector) unlinked.	Isolate IAM logic into an independent microservice; implement bounded working-memory injection under strict anti-drift constraints; systematically deploy remaining execution paths.
Implementation Hardening	RFQ ID generation vulnerable to high-concurrency race conditions (LG-06); UI component uncontainerized; manager-to-intelligence trigger remains explicit rather than autonomous.	Implement advisory-lock synchronization for transactional IDs; deploy UI via Docker containers; engineer a robust event-bus architecture to drive autonomous intelligence triggers.

Table 6.6: Consolidated register of current platform constraints and projected mitigations

The absence of a curated, empirical benchmark for evaluating probabilistic LLM accuracy represents the most acute technical vulnerability. Planned mitigations prioritize systemic observability and rigorous quantitative validation over lateral feature expansion.

CONCLUSION

This chapter has systematically validated the Itqan platform against the rigorous specifications established in Chapters 3 and 4. The evidence (comprising high-density pytest coverage, static anti-drift tests, and a bounded business-impact mapping) confirms the structural integrity of the trust-boundary architecture within the documented scope. The strategic pivot towards comprehensive lifecycle intelligence has yielded a robust, verifiable, and highly deterministic engineering platform, addressing key operational constraints while clearly demarcating the vector for future scaling.



GENERAL CONCLUSION

The work presented in this report set out to address a structural vulnerability observed during a five-day audit at Al Bassam Group: an estimation organization rich in individual expertise, yet constrained by disjointed connectivity, opaque traceability, and an inability to systematically leverage historical data for early feasibility assessments. These initial findings necessitated a strategic reframing of the original mandate. Rather than pursuing the automation of the Bill of Quantities, an objective deemed operationally unsafe given the asymmetric cost-of-error inherent to industrial estimation, the project pivoted to engineer a platform that fortifies deterministic workflows while preserving irreducible human expertise. The resulting platform, Itqan, forms the technical synthesis of this dissertation.

The Strategic Reframing. The pivot from localized automation to systemic RFQ Lifecycle Intelligence (Chapter 1) constitutes the foundational contribution: every subsequent architectural decision derives from this initial problem redefinition.

The Target Architecture. The Workflow and Tracking module is substantially delivered as the operational backbone. Communication Automation and Business Intelligence remain partially realized, evidenced by the reminder infrastructure and analytical aggregation endpoints. The Historical Learning layer is established via deterministic document extraction, with predictive modeling deferred to future iterations.

The Trust-Boundary Architecture. The copilot design delineated in Chapter 4 represents the central engineering innovation. It structurally isolates classification, data retrieval, and guardrail verification, unifying execution through a singular deterministic chokepoint (the ExecutionPlanFactory). Post-generation verification is enforced via deterministic guardrails

and a secondary language-model judge. These invariants are protected dynamically by anti-drift tests, distinguishing the deployment from a conceptual prototype.

The Distributed Implementation. Implementation spanned four cooperating microservices. The manager service enforces the lifecycle backbone via ACID-compliant stage gates; the copilot service deployed its first vertical slice (Paths 1, 4, and the robust Path 8 safety matrix); the intelligence service instantiated deterministic parser logic; and the user interface integrated these backend capabilities via structured, role-aware routing.

Empirical Validation. Extensive backend pytest coverage (259 manager tests, 636 copilot tests, and 64 fixture-independent intelligence tests), static anti-drift guardrails, dynamic pipeline validation, and a structured 2/7/3 business-impact mapping corroborate that the architecture operates as designed within the validated scope.

Limitations are documented transparently in Chapter 6. They predominantly manifest as *validation gaps* (e.g., CI execution scoped exclusively to the manager, testing targets restricted to SQLite, absent E2E browser automation) and *architectural deferrals* (e.g., dedicated IAM service, bounded working-memory prompt injection). Crucially, mitigating these limitations requires lateral expansion rather than foundational architectural refactoring.

Future development trajectories are explicit. **Short-term hardening** necessitates unified CI pipelines, PostgreSQL test integration, and the establishment of a rigorous benchmark for evaluating probabilistic LLM accuracy. **Medium-term objectives** involve deploying remaining copilot matrices (Paths 2, 3, 5, 6, 7), activating autonomous event-bus triggers for the intelligence service, and securely injecting working memory. **Long-term strategy** focuses on extracting the dedicated IAM service and deploying predictive analytics (Pillar 4) atop the newly acquired longitudinal dataset.

This project delivers a verifiable, robust platform whose core architectural innovation is rigorously grounded. It does not claim to have entirely eradicated the operational vulnerabilities at Al Bassam Group; rather, it provides the deterministic infrastructural backbone required to systematically address them under the rigorous engineering standards demanded by industrial estimation.



BIBLIOGRAPHY

- [1] A. Cockburn, “Hexagonal Architecture”, HaT Technical Report, 2005. Available: <https://alistair.cockburn.us/hexagonal-architecture/> (retrieved 2026-05-10).
- [2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, August 2021. ISBN: 978-1-492-03402-5.
- [3] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017. ISBN: 978-0-13-449416-6.
- [4] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, August 2003. ISBN: 978-0-321-12521-7.
- [5] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*, 2nd ed. Berlin, Germany: Springer, 2018. ISBN: 978-3-662-56508-7. DOI: <https://doi.org/10.1007/978-3-662-56509-4>.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. arXiv:2005.11401.
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [8] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan,

- R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. arXiv:2005.14165.
- [9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, D. Chen, W. Dai, H. S. Chan, A. Madotto, and P. Fung, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, article 248, March 2023. DOI: <https://doi.org/10.1145/3571730>. arXiv:2202.03629.
- [10] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. El Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan, “Constitutional AI: Harmlessness from AI Feedback,” Anthropic Technical Report, December 2022. arXiv:2212.08073.
- [11] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, vol. 36, 2023. arXiv:2306.05685.
- [12] Anthropic, “Tool use with Claude,” Claude API documentation, retrieved 2026-05-10. Available: <https://platform.claude.com/docs/en/docs/agents-and-tools/tool-use/overview>.
- [13] Anthropic, “Structured outputs,” Claude API documentation, retrieved 2026-05-10. Available: <https://platform.claude.com/docs/en/docs/build-with-claude/structured-outputs>.

- [14] SAP SE, “SAP CPQ, Configure, Price, Quote (CPQ) Solutions for Sales,” SAP product documentation, retrieved 2026-05-10. Available: <https://www.sap.com/products/financial-management/cpq.html>.
- [15] Salesforce, “Salesforce CPQ, Getting Started,” Salesforce Help (Spring '26 release, Version 66.0), retrieved 2026-05-10. Available: https://help.salesforce.com/s/articleView?id=sales.cpq_get_started.htm.
- [16] American Society of Mechanical Engineers (ASME), *BPVC.VIII.1-2025: Boiler and Pressure Vessel Code, Section VIII, Rules for Construction of Pressure Vessels, Division 1*, 2025 Edition. New York, NY, USA: ASME, 2025.
- [17] Tubular Exchanger Manufacturers Association (TEMA), *Standards of the Tubular Exchanger Manufacturers Association*, 11th Edition. Tarrytown, NY, USA: Tubular Exchanger Manufacturers Association, Inc.

A.1 Detailed Use Case Descriptions

This appendix expands the principal use cases introduced in Chapter 3 (Section 3.4) with structured descriptions covering actor, preconditions, main flow, alternate flows, and postconditions. Use cases related to target / future actors (Administrator template management, full executive analytics) are listed for completeness; current-scope implementation status is indicated per Chapter 3.

A.1.1 UC-01, Register RFQ

Primary actor: Estimation Manager. **Preconditions:** the user is authenticated; at least one workflow template is seeded in the manager service. **Main flow:** (1) the user opens the new-RFQ form; (2) provides metadata and selects a workflow template; (3) the system instantiates the RFQ, allocates a unique code, and creates per-stage instances; (4) the system persists and returns the RFQ. **Alternate flows:** (3a) validation failure; (3b) code-counter contention. **Postconditions:** RFQ exists with initial stage active and lifecycle timestamps recorded.

A.1.2 UC-02, Advance RFQ Stage

Primary actor: Estimator or Estimation Manager. **Preconditions:** the RFQ exists. **Main flow:** (1) user submits advance request; (2) system executes mandatory-field gate; (3) system executes blocker gate; (4) system executes transition-validity gate; (5) transaction commits atomically. **Alternate flows:** Gate failures yield structured constraint exceptions. **Postconditions:** RFQ advances, or lifecycle remains unchanged upon gate failure.

A.1.3 UC-03, Manage Stage Artifacts

Primary actor: Department User, Estimator, or Estimation Manager. **Preconditions:** appropriate role context. **Main flow:** (1) user attaches subtask, note, or file; (2) system validates

payload constraints; (3) subtask updates propagate. **Alternate flows:** (1b) edit attempt on append-only note refused. **Postconditions:** artifact persisted; traceability updated.

A.1.4 UC-04, Configure Reminders

Primary actor: Estimation Manager. **Preconditions:** target exists. **Main flow:** (1) user creates reminder; (2) system persists reminder parameters; (3) reminder flagged for processing. **Postconditions:** reminder persisted; outbound dispatch deferred.

A.1.5 UC-05, Consult Pipeline Analytics

Primary actor: Executive. **Preconditions:** authenticated dashboard access. **Main flow:** (1) user requests dashboard; (2) UI consumes analytical endpoints; (3) KPIs rendered. **Alternate flows:** null financial values suppressed. **Postconditions:** read-only access logged.

A.1.6 UC-06, Query Copilot in Portfolio Mode

Primary actor: any authorised business actor (Estimation Manager, Estimator, or Executive) operating the copilot in portfolio mode. **Preconditions:** authenticated. **Main flow:** (1) user submits turn; (2) pipeline classifies intent; (3) ExecutionPlanFactory orchestrates deterministic retrieval, context building, and guardrails; (4) response delivered. **Alternate flows:** failure routes to Path 8. **Postconditions:** turn forensics persisted.

A.1.7 UC-07, Query Copilot in RFQ-Bound Mode

Primary actor: an RFQ-authorised actor (Estimation Manager or Estimator). **Preconditions:** RFQ selected. **Main flow:** as UC-06, primarily executing Path 4. **Postconditions:** evidence anchored to target RFQ.

A.1.8 UC-08, Manage Workflow Templates

Primary actor: Administrator. **Preconditions:** administrative context. **Main flow:** (1) user inspects template metadata. **Alternate flows:** schema edits deferred to external tooling. **Postconditions:** catalog consistency maintained.

A.2 Product Backlog

The product backlog maps each functional requirement to an operational item, prioritized by strategic urgency and classified by implementation status.

Table A.1: Product backlog mapping functional requirements to current statuses

Module	ID	Item	Priority	Status
Manager	FR-MGR-01	Register new RFQ with metadata and workflow template	High	Done
Manager	FR-MGR-02	Instantiate per-RFQ stages from workflow template	High	Done
Manager	FR-MGR-03	Three-gate stage progression (mandatory / blocker / transition)	High	Done
Manager	FR-MGR-04	Attach files, notes, and subtasks to stages (append-only notes)	High	Done
Manager	FR-MGR-05	Manual and rule-based reminders with assignee and due date	High	Done
Manager	FR-MGR-06	Analytical endpoints (status totals, win rate, per-client)	High	In Progress
Manager	FR-MGR-07	Lifecycle traceability via timestamps and non-destructive updates	High	Done
Manager	FR-MGR-08	Soft delete of subtasks with progress propagation	High	Done
Copilot	FR-COP-01	Natural-language queries with evidence-grounded responses	High	Done
Copilot	FR-COP-02	Portfolio-mode and RFQ-bound-mode entry points	High	Done
Copilot	FR-COP-03	Turn classification into a known interaction path	High	Done
Copilot	FR-COP-04	Path-authorised evidence grounding	High	In Progress
Copilot	FR-COP-05	Structured refusal for out-of-scope queries (Path 8 family)	High	Done
Copilot	FR-COP-06	Manager-mediated access enforcement on every retrieval	High	Done

continued on next page

Table A.1 - continued from previous page

Module	ID	Item	Priority	Status
Copilot	FR-COP-07	Bounded conversation history and working-memory capture	High	In Progress
Copilot	FR-COP-08	Response verification against retrieved evidence before delivery	High	Done
Intelligence	FR-INT-01	MR-package parser with deterministic profile output	Medium	Done
Intelligence	FR-INT-02	Workbook parser with identity / item / cost / schedule profiles	Medium	Done
Intelligence	FR-INT-03	Intelligence briefing for downstream presentation	Medium	In Progress
Intelligence	FR-INT-04	Anomaly and cross-check flags rather than silent corrections	Medium	Done
UI	FR-UI-01	Portfolio view with filter/sort attributes	High	Done
UI	FR-UI-02	RFQ detail view with lifecycle, artifacts, and copilot entry point	High	Done
UI	FR-UI-03	Portfolio-level executive indicators and role-aware presentation	High	In Progress
Cross-cutting	FR-X-01	Independently containerised and startable backend services	Medium	In Progress
Cross-cutting	FR-X-02	Health endpoint and metrics surface where implemented	Medium	In Progress

A.3 Path Registry Structure

The Path Registry encodes the entire copilot policy surface as structured configuration. Key vectors include `intent_topics`, `resolver_strategy`, `access_policy`, `forbidden_fields`, `memory_policy`, and `active_guardrails`. The registry currently operationalizes Path 1, Path 4, and the Path 8 exception matrix, validating that new execution capabilities require only declarative configuration updates rather than low-level pipeline refactoring.

A.4 Anti-Drift Tests and Test Invocation Evidence

This appendix catalogues the specific anti-drift files and pytest invocation outcomes referenced in Chapter 6.

A.4.1 Anti-Drift Test Files (Copilot Service)

18 assertions are enforced across eight files located at `microservices/rfq_copilot_ms/tests/anti_drift/`:

- `test_no_turn_execution_plan_outside_factory.py`: Enforces single-construction invariant via AST analysis.
- `test_path_registry_reader_allowlist.py`: Restricts policy access to the Factory and Escalation Gate.
- `test_fast_intake_anchored.py`: Validates strict regex anchoring without external latency.
- `test_llm_structured_output_enforced.py`: Enforces Pydantic schema validation on LLM responses.
- `test_no_llm_sdk_in_deterministic_stages.py`: Guarantees execution purity in non-generative stages.
- `test_memory_policy_load_bearing.py`: Asserts configuration parameters are actively evaluated.
- `test_no_v3_references.py`: Eradicates superseded architectural references.
- `test_batch10_no_history_injection.py`: Validates strict capture-only memory enforcement.

A.4.2 Test Invocations and Pass Counts

Pytest executions run via `python -m pytest -q`.

Microservice	Execution Outcome	Latency
rfq_manager_ms	259 passed, 121 warnings	22.87 s
rfq_copilot_ms	636 passed, 3 warnings	6.24 s
rfq_intelligence_ms	64 passed, 48 fixture-dependent, 6 skipped, 2 warnings	2.72 s

Table A.2: Automated pytest execution metrics (Commit 72d5ae2, Python 3.11.9)

A.4.3 Frontend Source-Contract Scripts

Twelve source-contract validation scripts execute via Node.js strict-mode. Eleven pass; one (`p0-truth-gates.test.mjs`) demonstrates an acceptable assertion drift currently tagged for alignment.

A.5 Golden Scenario Test Set

The copilot suite utilizes pipeline and smoke tests as a proxy for a curated golden dataset. The 216 isolated pipeline functions and 98 end-to-end traversal scenarios extensively model Path 1, Path 4, and Path 8 behaviors. Constructing an independent, LLM-evaluated golden benchmark represents the primary future validation objective.

A.6 Postman Validation Structure

The manager service includes a comprehensive Postman collection categorized into Initialization, Happy-Path Walkthrough, Advanced Guardrails, and Read-Only Diagnostics. These scripts validate atomic database state manipulation, negative assertion rejection, and accurate KPI aggregation. Automated integration via Newman is pending.

A.7 Integration Paths

Integration Vector	Status	Validation Method
UI to Manager	Validated	Contract scripts and deterministic UI routing.
UI to Copilot	Validated	Real-time validation of thread and turn payload schemas.
UI to Intelligence	Validated	Artifact visualization validation.
Copilot to Manager	Stubbed/Validated	Mocked for unit analysis; validated via manual integration testing.
Manager to Intelligence	Manual Trigger	Explicit API triggers verified; event-bus integration deferred.
Intelligence to Manager	Validated	Deterministic read-only consumption established.
Copilot to Intelligence	Pending	Architecturally prepared as Path 5 execution.

Table A.3: Cross-service integration mapping and validation maturity

A.8 Business Impact Mapping

Table A.4: Extensive mapping of audit vulnerabilities to implemented capabilities

ID	Vulnerability	Status	Resolution Vector
PP-01	Inter-departmental silos	Addressed	Unified RFQ lifecycle indexing and stage metrics.
PP-02	Manual follow-up overhead	Partially addressed	Internal reminder architecture executed; outbound webhooks pending.
PP-03	Zero executive visibility	Partially addressed	Aggregated APIs instantiated; predictive financial metrics deferred.
PP-04	Absence of historical reuse	Not yet addressed	Lifecycle persistence is active; historical retrieval and reuse algorithms are deferred.
PP-05	Late infeasibility discovery	Partially addressed	Blocker gate intercepts issues; deep pattern analysis pending.
PP-06	Workbook structural fragility	Partially addressed	Algorithmic extraction active; original workbook remains primary.
PP-07	Undocumented expertise	Partially addressed	Structured note capture is supported; advanced expertise reuse remains deferred.
PP-08	Non-compliant classification	Addressed	Transition-validity gate prohibits unauthorized workflows.
PP-09	Unmanaged supplier risk	Not yet addressed	Requires Pillar 4 predictive intelligence integration.
PP-10	ERP isolation	Not yet addressed	Acknowledged entirely as future scope.

continued on next page

Table A.4 - continued from previous page

ID	Vulnerability	Status	Resolution Vector
PP-11	Intuition-based prioritization	Partially addressed	Basic win/loss aggregation is available; prioritization heuristics and multi-variate modeling are deferred.
PP-12	Standards cost isolation	Partially addressed	Deterministic extraction foundations are available; cost-model integration remains pending.

A.9 Limitations Register

Table A.5: Register of architectural limitations and mitigation trajectories

Domain	Operational Constraint	Hardening Trajectory
Validation	Testing limited to SQLite and backend components.	PostgreSQL integration and full UI E2E assertion.
Concurrency	Race conditions possible during ID generation.	Integration of advisory-lock mechanisms.
Evaluation	Probabilistic model evaluation relies on deterministic test doubles.	Construction of rigorous, reproducible golden test benchmarks.
Intelligence	Parsing restricted to local proprietary assets.	Creation of reproducible, sanitized fixture data sets.
Integration	Reliance on explicit HTTP triggers between Manager and Intelligence.	Event-bus paradigm shift for autonomous cascading.
Access	Monolithic IAM shim embedded in manager.	Microservice extraction to standalone OIDC provider.
Memory	Captured working memory not recurrently evaluated.	Systematic injection under strict architectural boundaries.

A.10 Architectural Decisions Register

Vector	Analyzed Alternatives	Adopted Strategy	Engineering Rationale	Accepted Trade-off
Topology	Monolith, Microservices	Microservices	Isolation of failure domains and scalable capability augmentation.	Elevated integration complexity and latency overhead.
Design Pattern	MVC, Hexagonal, BACAB	BACAB layered	Guaranteed structural consistency across backend implementations.	Strict boilerplate enforcement on low-complexity modules.
Data Authority	Distributed, Manager-owned	Manager-owned	Absolute guarantee of a single point of truth for transactional states.	Creates a systemic single point of failure and bottleneck.
Conversational Engine	Generic LLM, Naive RAG, Trust-Bound Copilot	Trust-Bound Copilot	Eliminates probabilistic hallucination vulnerabilities via deterministic oversight.	Exponentially increases architectural complexity over standard RAG.
Policy Enforcement	Hard-coded logic, Declarative Configuration	Declarative Configuration	Decouples capability expansion from core source code modifications.	Configuration drift presents significant operational risk.
Orchestration	Fragmented logic, Singular Factory	ExecutionPlanFactory	Guarantees anti-drift validation and strict runtime policing.	Forces rigid compliance across all novel query intents.

Table A.6: Consolidated registry of foundational architectural decisions

Itqān: An RFQ Lifecycle Intelligence Platform with a Bounded AI Copilot

Mohamed Guidara

Abstract :

This dissertation presents the design and implementation of Itqan, a data engineering platform developed to streamline the Request for Quotation (RFQ) lifecycle within an industrial estimation context. Addressing the limitations of localized automation and severe communication silos, the project introduces a systemic architectural solution. The core innovation is a conversational copilot governed by strict deterministic trust boundaries, which safeguards operational truth and neutralizes vulnerabilities inherent to probabilistic language models. Deployed as four cooperating microservices, with its three backend services structured under the BACAB pattern, the platform delivers a resilient transactional backbone. Validated through extensive backend automated testing within a documented evidence scope, it establishes the foundational infrastructure required for future predictive analytics capabilities.

Key Words : Data Engineering, Business Intelligence, Microservices, RFQ Lifecycle, Trust Boundaries

Résumé :

Ce mémoire détaille la conception et l'implémentation d'Itqan, une plateforme d'ingénierie des données conçue pour rationaliser le cycle de vie des appels d'offres (RFQ) dans un environnement industriel. Face aux limites de l'automatisation locale et aux silos de communication, le projet propose une architecture systémique. L'innovation majeure réside dans un copilote conversationnel structuré par de strictes frontières de confiance déterministes. Cette approche garantit l'exactitude opérationnelle et neutralise les vulnérabilités inhérentes aux modèles de langage probabilistes. Déployée via quatre microservices coopérants, dont les trois services backend suivent le modèle BACAB, la plateforme établit un socle transactionnel résilient, validé par des tests automatisés étendus côté backend, posant ainsi l'infrastructure nécessaire aux futures capacités analytiques prédictives.

Mots Clés : Ingénierie des Données, Systèmes Décisionnels, Microservices, Cycle de vie RFQ, Frontières de Confiance

تلخيص:

يعرض هذا التقرير تصميم وتنفيذ منصة إيتقان، وهي منصة هندسة بيانات موجهة لتحسين دورة حياة طلبات عروض الأسعار في سياق التقدير الصناعي. يعالج المشروع ضعف التواصل والتتبع ومحدودية الأتمتة المحلية عبر بنية خدمات مصغرة تفصل بين الحقيقة التشغيلية وقدرات النماذج اللغوية. ويرتكز الابتكار الأساسي على مساعد محادثة محكوم بحدود ثقة حتمية تحفظ سلامة القرارات ودور الخبرة البشرية. وقد تم التحقق من المنصة ضمن نطاق أدلة موثق، مع تمهيد الطريق لقدرات تحليلية وتنبؤية مستقبلية.

الكلمات المفتاحية: هندسة البيانات، ذكاء الأعمال، الخدمات المصغرة، دورة حياة طلبات العروض، حدود الثقة